AKS / ARCHITECTURE FIELD GUIDE

The Azure Kubernetes Service Checklist

A deliberate approach to production readiness.

Review the decisions that matter. Build resilient foundations, protect workloads and operate with evidence.

Content reviewed: 2026-11-04 / English edition

178	11	02
Review controls	Architecture domains	Unscored references

An independent architecture review aid. Not a certification or a guarantee of production readiness.

START HERE

Review outcomes, not feature adoption.

An architecture review aid, not a certification or a guarantee of production readiness. Apply checks according to workload requirements, risk, region and service support. Record exceptions and evidence. Verify current feature availability, retirement notices and pricing in the linked documentation before implementation. Conditional checks require an applicability decision; record Not applicable separately from Fulfilled. Reference guidance is not scored.

01 / Establish applicability

Read the scope of each control. Choose implementations that fit the workload, risk and operating model.

02 / Capture evidence

Record the configuration, observations and recovery tests that demonstrate the intended outcome.

03 / Make a clear decision

Distinguish Fulfilled, Open and Not applicable. Assign an owner to exceptions and revisit them as the platform changes.

Priority guide High Medium Low Review priority, not an automatic compliance verdict.

EXPLORE THE GUIDE

Contents

11 architecture domains. One coherent review.

01 **Identity - Authorization**
11 review controls

02 **Cluster security**
18 review controls

03 **Multi-Tenant - Isolation**
4 review controls

04 **Storage**
12 review controls

05 **Networking**
25 review controls

06 **Resource Management**
15 review controls

07 **Cluster Operations**
33 review controls

08 **Business continuity - disaster recovery**
15 review controls

09 **Windows**
14 review controls

10 **Application deployment**
22 review controls

11 **Image management**
9 review controls

A **Reference guidance (not scored)**
2 supporting references

ARCHITECTURE DOMAIN 01 / 11

Identity - Authorization

Establish trusted identities and deliberately scoped access for people, clusters and workloads.

11 review controls / 4 high / 4 medium / 3 low

Control 01.01

Priority: High

Use AKS-managed Microsoft Entra ID authentication

Authenticate users to the Kubernetes API with AKS-managed Microsoft Entra ID integration and group-based access. Migrate legacy integration and validate administrator group access before disabling local accounts. Authentication establishes identity; configure Kubernetes or Azure RBAC separately for authorization.

DOCUMENTATION

- Microsoft Entra ID control plane authentication
- Disable local accounts

Control 01.02

Priority: High

Choose and scope Kubernetes RBAC or Azure RBAC for Kubernetes authorization

Use Microsoft Entra groups with Kubernetes Roles/RoleBindings, or Azure RBAC for Kubernetes Authorization, to grant least-privilege access at namespace scope where possible. Distinguish Kubernetes data-plane permissions from Azure management-plane roles that retrieve kubeconfig or change the cluster. Review cluster-admin bindings and privilege-escalation paths regularly.

DOCUMENTATION

- Microsoft Entra ID with Kubernetes RBAC
- Azure RBAC for Kubernetes authorization

Control 01.03

Priority: Medium

Use AKS and ACR integration without password

Use the kubelet managed identity to pull images from Azure Container Registry instead of registry admin credentials or long-lived image pull secrets. Grant only the required registry or repository access.

For RBAC-only registries, AKS ACR integration assigns AcrPull to the kubelet identity. For registries using RBAC Registry + ABAC Repository Permissions, --attach-acr is not supported: assign Container Registry Repository Reader manually, with repository conditions where appropriate. Workload identity for application Azure API calls is separate from kubelet image-pull authentication.

DOCUMENTATION

- Authenticate with Azure Container Registry from AKS

Control 01.04

Priority: High

Use least-privilege managed identities for cluster and kubelet access

Prefer system-assigned or user-assigned managed identities over cluster service principals to avoid credential rotation. Separate control-plane resource management permissions from kubelet image-pull permissions, scope grants to required resources and do not reuse node identities for application access. During identity migration, retain necessary old grants until token refresh and validation complete.

Review effective Azure role assignments, scopes and inherited grants for the actual control-plane and kubelet identities, including network and registry access. Confirm representative operations succeed and unauthorized operations are denied; a managed-identity flag alone does not demonstrate least privilege.

DOCUMENTATION

- Use managed identities in Azure Kubernetes Service

Control 01.05

Priority: Medium

Limit access to admin kubeconfig (get-credentials --admin)

Restrict Azure permissions to retrieve cluster-admin credentials, update identity settings, invoke AKS Run Command or manage nodes. Prefer user kubeconfig with Microsoft Entra authentication; retrieving user kubeconfig does not itself grant Kubernetes authorization. Audit privileged management operations and keep an approved, tested emergency access procedure.

DOCUMENTATION

- Use Azure RBAC to define access to the Kubernetes configuration file

Control 01.06

Priority: Medium

For AKS non-interactive logins use kubelogin

Use kubelogin's supported exec authentication modes for automation. Prefer managed identity on Azure-hosted agents or federated workload identity for CI over service principal secrets. Grant the automation identity only required cluster permissions and ensure it has network access to the API server; avoid interactive/device-code flows in pipelines.

DOCUMENTATION

- Non-interactive sign in with kubelogin

Control 01.07

Priority: Medium

Disable AKS local accounts

Disable local accounts after validating Microsoft Entra administrator and automation access so static administrator credentials cannot bypass identity controls. On existing clusters, rotate cluster certificates to revoke previously issued local credentials. Document tightly controlled emergency re-enablement, audit its use and disable local accounts again after recovery.

DOCUMENTATION

- Disable local accounts

Control 01.08

Priority: Low

Use Privileged Identity Management for just-in-time privileged access

Use Microsoft Entra Privileged Identity Management for eligible, time-limited administrative role or group activation with approval and MFA as appropriate. Scope access separately for Kubernetes administration and node access, review activations and account for Microsoft Entra licensing requirements.

DOCUMENTATION

- Control AKS cluster and node access with PIM

Control 01.09

Priority: Low

Precreate a user-assigned kubelet identity when permissions are needed before provisioning

Use a precreated user-assigned kubelet identity when registry permissions or group membership must exist before cluster creation. Grant the control-plane identity the required permission to assign it, especially outside the node resource group, and allow role propagation time. A kubelet identity is not a pod workload identity.

DOCUMENTATION

- AKS managed identity types and authorization

Control 01.10

Priority: Low

Apply Microsoft Entra Conditional Access to human cluster sign-ins

Apply appropriate MFA, device and location requirements to Microsoft Entra-authenticated administrator sign-ins. Validate kubelogin authentication flows and emergency access before enforcement, and review licensing. Do not assume user Conditional Access policies protect local certificate credentials or workload identities.

DOCUMENTATION

- Microsoft Entra Conditional Access overview
- AKS kubelogin authentication methods

Control 01.11

Priority: High

Use Microsoft Entra Workload ID and migrate retired pod-managed identity

> **Applicability:** Applies to pods requiring supported Azure service authentication, including deployments migrating from retired pod-managed identity.

Use Microsoft Entra Workload ID for pod-to-Azure authentication without stored credentials. Azure AD Pod Identity was deprecated on October 24, 2022 and its project archived in September 2023; support for the AKS managed add-on ran through September 2025. Migrate remaining bindings and applications instead of deploying it.

Enable the OIDC issuer and workload identity on AKS Standard (preconfigured on AKS Automatic). Configure dedicated Kubernetes service accounts, exact federated issuer/namespace/service-account subject/audience bindings, the azure.workload.identity/use: true pod label and supported Azure Identity SDK or MSAL versions. Grant only required Azure data-plane permissions per workload, test token renewal and remove old pod identity components and node identity grants only after successful migration.

DOCUMENTATION

- Microsoft Entra Workload ID
- Migrate from pod-managed identity
- Pod-managed identity deprecation and support dates
- Deploy and configure Microsoft Entra Workload ID

ARCHITECTURE DOMAIN 02 / 11

Cluster security

Build layered protection around the control plane, nodes, secrets and policy boundaries.

18 review controls / 8 high / 7 medium / 3 low

Control 02.01 Priority: Medium

Configure your cluster for regulated industries

> **Applicability:** Applies to workloads subject to specific regulatory or cryptographic compliance requirements.

Map applicable regulatory requirements to AKS controls and evidence, including supported FIPS-enabled node images, encryption, identity, network isolation and audit retention. Validate OS/Kubernetes/region compatibility and the actual cryptographic module certification; enabling FIPS or selecting an architecture does not automatically make the workload compliant.

DOCUMENTATION

- Use FIPS-enabled node pool
- AKS CIS benchmark
- AKS PCI DSS 4.0.1 architecture guidance

Control 02.02 Priority: Medium

Remove unnecessary dashboard deployments and restrict administration UIs

> **Applicability:** Applies when legacy dashboard resources or other cluster administration UIs are installed.

The historical AKS-managed Kubernetes dashboard add-on is no longer a supported administration path. Inventory and remove unnecessary legacy or self-installed dashboard resources, service accounts and privileged bindings. For required administration UIs, use supported tooling with authenticated, least-privilege access and no anonymous public endpoint.

Use the Azure portal Kubernetes resource views or kubectl with Microsoft Entra authentication and the required network connectivity. Do not rely on obsolete kube-dashboard add-on commands to remove independently installed dashboards.

DOCUMENTATION

- Access Kubernetes resources using the Azure portal

Control 02.03

Priority: Low

Select Kubernetes secret encryption and key lifecycle controls

> **Applicability:** Applies when requirements mandate customer-controlled keys for Kubernetes secrets in etcd, or an existing KMS configuration must be maintained.

AKS encrypts stored data at rest with Microsoft-managed keys by default. Where requirements mandate customer-controlled keys for Kubernetes secrets in etcd, evaluate the supported Azure Key Vault KMS configuration and its version, identity and network prerequisites. This is separate from node disk and application persistent-volume encryption.

The newer KMS data encryption experience for Kubernetes 1.33 or later is currently documented as preview; do not treat it as an unconditional production baseline or assume migration from legacy KMS is automatic. For the selected experience, protect keys with soft delete/purge protection, least-privilege access and tested rotation/recovery. Loss of required keys can make secrets unreadable.

DOCUMENTATION

- Add KMS etcd encryption to an Azure Kubernetes Service (AKS) cluster
- KMS data encryption (preview)

Control 02.04

Priority: High

Validate Kubernetes API and version compatibility before support expires

Inventory Kubernetes API versions, deprecated or removed APIs, add-ons, CRDs, admission webhooks and client compatibility against the current and target AKS versions. Track the release calendar, regional upgrade targets, supported patches and control-plane/node version skew so compatibility validation and minor-version upgrades complete before support expires.

Standard community support generally covers three minor versions; Premium LTS has separate eligibility and patch requirements. Retain rendered-manifest validation and representative upgrade-test evidence, including controller reconciliation after API changes. This control establishes Kubernetes compatibility; configure the upgrade mechanism and node-image or OS lifecycle controls separately.

DOCUMENTATION

- Regularly update to the latest version of Kubernetes
- Kubernetes API deprecation guide
- Supported Kubernetes versions and LTS

Control 02.05

Priority: Medium

Gate image deployment on vulnerability findings

Scan images in CI and supported registries, define vulnerability thresholds and enforce them at admission using a supported policy or Defender for Containers gated deployment. Validate prerequisites and support for the target cloud, cluster and registry. Begin with audit, then enable deny with controlled exemptions and defined behavior for missing or stale scan findings.

DOCUMENTATION

- Defender for Containers gated deployment

Control 02.06

Priority: High

Manage application secrets with least-privilege access and tested rotation

Keep credentials out of images, manifests, build arguments and source control. Manage application secrets, keys and certificates with Azure Key Vault using Microsoft Entra Workload ID and least-privilege access through the supported Secrets Store CSI Driver or application SDK. Demonstrate rotation and application reload without exposing secret contents.

Kubernetes secrets are encrypted at rest by AKS, but base64 encoding is not encryption and authorized API access can reveal values. Limit synchronization into Kubernetes, restrict secret-read permissions and avoid logging contents. Environment variables require a pod restart to pick up changes and subPath mounts do not receive automatic updates. Key Vault is a secrets management service, not synonymous with a dedicated HSM.

DOCUMENTATION

- Azure Key Vault provider for Secrets Store CSI Driver
- Secrets Store CSI Driver configuration and rotation

Control 02.07

Priority: High

Clean unused vulnerable node images and remediate running workloads separately

Use Image Cleaner to remove unused images with vulnerabilities from supported AKS Linux nodes. It does not patch images, remove images in use, or clean the registry; rebuild and redeploy vulnerable workloads separately.

Image Cleaner is preconfigured on AKS Automatic and optional on AKS Standard. Set an appropriate cleanup interval, monitor cleanup failures and required scanner connectivity, and account for its lack of support for Windows node pools and virtual nodes.

DOCUMENTATION

- Use ImageCleaner to clean up stale images on your AKS cluster

Control 02.08

Priority: High

Detect runtime threats and exercise incident response

Operate supported runtime threat detection for Kubernetes, nodes and applications, such as Microsoft Defender for Containers or a maintained equivalent. Verify sensor health and alert ingestion, route actionable alerts to owned responders, and exercise triage, containment and recovery. Subscription enablement or sensor installation alone is not evidence of effective detection and response.

For the chosen implementation, validate plan/components, pricing and coverage for the cloud, region, OS, architecture and private-network dependencies. Tune detections and use safe test events to confirm delivery and response runbooks. Runtime detection complements preventive hardening, image scanning, admission enforcement and posture review; it does not guarantee that attacks are blocked.

DOCUMENTATION

- Introduction to Microsoft Defender for Containers
- Defender for Containers deployment overview

TOOLS

- Falco

Control 02.09

Priority: Medium

Verify centralized Kubernetes policy enforcement and exception ownership

Use the supported Azure Policy add-on and Kubernetes policy initiatives for centralized audit and admission enforcement. Roll policies out in audit before deny, validate system/add-on compatibility and document narrowly scoped exceptions. Monitor policy health and compliance; assigning an initiative does not automatically remediate existing workloads.

Retain effective assignments, constraint status, compliance results and exception owners/expiry as evidence. Verify enforcement with safe allowed and denied test manifests in a representative namespace, and track remediation of existing violations. An enabled add-on or assigned initiative alone cannot prove that the intended workloads are covered or that admission denial works.

DOCUMENTATION

- Azure Policies for AKS
- Azure Kubernetes Services Governance with Azure Policy

TOOLS

- Gatekeeper

Control 02.10

Priority: Medium

Separate application workloads from system components with dedicated node pools

Run cluster-critical system components on appropriately sized system pools and application workloads on user pools. Where supported, use CriticalAddonsOnly=true:NoSchedule to keep ordinary workloads off system pools and ensure system replicas have sufficient capacity. The AKS control plane is already Microsoft-managed and does not run on these node pools; pool separation is not a hard tenant security boundary.

A production cluster with a single system pool requires at least two nodes; prefer at least three, distributed across supported availability zones. System pools require Linux and VM SKUs with at least 4 vCPUs and 4 GB of memory; B-series and Spot aren't supported. Size above these minima for add-ons, DNS load, failover and surge upgrades, and test that critical pods can reschedule during disruption. Verify actual pod placement, taints and tolerations; the existence of system and user pools alone does not prove separation.

DOCUMENTATION

- AKS System Pools

Control 02.11

Priority: High

Rotate credentials for remaining service-principal-based clusters

> **Applicability:** Applies only to existing clusters still using a service principal; managed-identity clusters do not need service-principal credential rotation.

Prefer migrating cluster service principals to managed identities. Until migration, inventory credential expiry, alert before expiration and rotate using the supported AKS procedure at the organization's required interval. Validate permissions and cluster operations after rotation; do not embed credentials in manifests or pipelines.

DOCUMENTATION

- Update or rotate the credentials for AKS

Control 02.12

Priority: High

Use a private registry for your images, such as ACR

Use an authenticated registry such as Azure Container Registry for approved images and artifacts. Restrict pull/push permissions separately, disable unnecessary admin credentials, scan and patch images, and use immutable digests for deployments. A private registry requires authentication but does not automatically have private network connectivity.

DOCUMENTATION

- Azure Container Registry documentation

Control 02.13 Priority: Medium

Review Microsoft Defender for Cloud posture and vulnerability recommendations

Review Defender for Cloud recommendations for AKS configuration risks and container vulnerabilities, prioritize exploitable findings, and track remediation owners and deadlines. Validate posture coverage for the enabled plan and target environment; posture assessment complements rather than replaces runtime detection and admission enforcement.

DOCUMENTATION

- Microsoft Defender for Containers capabilities

Control 02.14 Priority: Low

If required consider using Confidential Compute for AKS

> **Applicability:** Applies when a documented threat model requires confidential computing or existing confidential workloads must be operated.

Select the confidential computing model that matches the threat model: application enclaves, confidential VM node pools and confidential containers have different isolation and attestation boundaries. Ordinary containers on a confidential VM still share its guest kernel. Validate supported regions, VM sizes, OS, Kubernetes versions and preview status for the specific feature before committing to it.

DOCUMENTATION

- Application enclave support on AKS

Control 02.15 Priority: Low

If required consider using Azure Dedicated Hosts for AKS nodes

> **Applicability:** Applies when physical server isolation or licensing requirements justify Dedicated Hosts, or existing AKS node pools use them.

Evaluate Azure Dedicated Host only when physical server isolation or licensing requirements justify it. Check supported VM families, host capacity, zone placement and AKS restrictions. Dedicated Hosts isolate the underlying hardware from other Azure customers; they do not isolate applications within your cluster.

DOCUMENTATION

- Add Azure Dedicated Host to an AKS cluster

Control 02.16

Priority: High

Enforce Pod Security Standards with controlled namespace exceptions

Use Pod Security Admission or equivalent Azure Policy controls to enforce an appropriate baseline, preferably Restricted for compatible application namespaces. PSA is enabled in AKS, but namespace enforcement labels must be configured; enabled admission alone is not proof of enforcement.

Start with audit/warn, test compatibility, then enforce and pin policy versions deliberately. Restrict who can change namespace security labels. Document exceptions for system components and workloads requiring host access; deny unnecessary privileged containers, host namespaces and privilege escalation, and apply non-root and seccomp controls where supported.

DOCUMENTATION

- Use Pod Security Admission in AKS
- Enforce Pod Security Standards with namespace labels

Control 02.17

Priority: High

Collect and protect Kubernetes audit and Azure management logs

Configure AKS diagnostic settings to retain the required control-plane logs, including kube-audit or kube-audit-admin as appropriate, and retain Azure Activity Logs for cluster management operations. Container stdout logs alone do not provide a Kubernetes audit trail.

Select audit and operational control-plane categories deliberately to balance read-event coverage, investigation needs, sensitive data exposure and cost. Configure approved destinations, resource-specific tables, retention and access controls; diagnostic settings do not backfill historical logs. Evaluate kube-audit-admin's reduced scope and table-plan query/alert limitations before filtering. Alert on privileged role changes, secret access, local-account re-enablement and diagnostic-setting changes, and validate ingestion and incident queries regularly.

DOCUMENTATION

- Monitor AKS and configure resource logs
- Enable and review Kubernetes control-plane logs

Control 02.18

Priority: Medium

Enable Trusted Launch for compatible node pools

Use Trusted Launch with Secure Boot and vTPM where the selected generation 2 VM, OS and workload support it. Validate custom drivers, GPU and other feature compatibility before enforcement, and monitor boot-integrity alerts. Trusted Launch protects the boot chain; it does not replace workload isolation or patching.

DOCUMENTATION

- Trusted Launch for AKS nodes

ARCHITECTURE DOMAIN 03 / 11

Multi-Tenant - Isolation

Choose isolation boundaries that reflect tenant trust, operational ownership and risk.

4 review controls / 2 high / 2 medium / 0 low

Control 03.01

Priority: Medium

Use logical isolation for mutually trusted tenants with explicit controls

> **Applicability:** Shared-cluster deployments with compatible, mutually trusted tenants whose trust, regulatory, lifecycle, and availability requirements permit logical isolation.

Consolidate compatible, mutually trusted teams with namespaces, scoped RBAC, network policies, quotas, and admission controls when shared-cluster economics justify the shared failure domain. Namespaces alone don't isolate tenants securely.

Document cluster-scoped resources, operator permissions, noisy-neighbor limits, and platform change ownership. Don't treat separate node pools as a hard security boundary or use logical isolation alone for hostile tenants.

RELATED CONTROLS

- Use namespaces with explicit access and isolation controls
- Use separate clusters where security, lifecycle, or failure boundaries require them

DOCUMENTATION

- Isolating cluster

Control 03.02

Priority: High

Use separate clusters where security, lifecycle, or failure boundaries require them

Document application and tenant trust, regulatory, lifecycle, platform dependency, availability, and upgrade boundaries before choosing namespace, node pool, or cluster separation. Use separate clusters for mutually untrusted tenants or requirements that need a stronger boundary. Record the decision and balance duplicated system capacity, operational overhead, and cost against the required isolation.

Namespaces require scoped RBAC, network policy, admission controls, and quotas; implement those controls separately when logical isolation is appropriate. Node pools share the Kubernetes API and are not a hard security boundary. Separate clusters still need independent identity, network, policy, and data-access controls. Regional resilience also requires application traffic routing, data replication, and tested failover; deploying a second cluster alone doesn't provide disaster recovery.

RELATED CONTROLS

- Use namespaces with explicit access and isolation controls

DOCUMENTATION

- Isolating cluster

Control 03.03

Priority: High

Stage multi-cluster upgrades with Azure Kubernetes Fleet Manager

> **Applicability:** Only estates with multiple AKS clusters that require coordinated update promotion; Fleet is an implementation choice, not a requirement for every cluster.

For multiple AKS clusters, evaluate Fleet update runs and reusable strategies to promote Kubernetes or node image updates through test, canary, and production stages. Use auto-upgrade profiles when automatic creation of update runs fits the operating model.

Fleet update runs honor member maintenance windows. Choose Latest images for freshness or Consistent images available across participating regions for repeatable testing. Coordinate Fleet with cluster auto-upgrade policies so independent updates don't bypass your intended rollout order. Approval gates, label-based groups, concurrency and failure thresholds, and the SecurityPatch channel are preview features in the current documentation; don't assume all orchestration controls have the same support status.

DOCUMENTATION

- Fleet update orchestration concepts and preview controls
- Configure Fleet update runs

Control 03.04

Priority: Medium

Choose Fleet capabilities and hub architecture for the actual multi-cluster need

> **Applicability:** Only multi-cluster estates evaluating Fleet for a defined update, placement, or governance need; single-cluster deployments and estates without that need are excluded.

Fleet can coordinate AKS updates without a hub cluster; Kubernetes resource placement requires a hub. Evaluate whether centralized placement and governance justify the additional control plane, permissions, and operating model instead of adding a hub solely to orchestrate upgrades.

Use cluster labels and placement policies deliberately and separate resource deployment from traffic routing and state replication. Supported operations differ by member cluster type. Fleet automated deployments, DNS-based load balancing, and cross-cluster networking are documented as preview; assess their support terms independently rather than treating Fleet as one uniformly GA feature.

DOCUMENTATION

- Azure Kubernetes Fleet Manager overview
- Fleet member cluster types and supported capabilities
- Fleet resource placement

ARCHITECTURE DOMAIN 04 / 11

Storage

Match persistent storage to application performance, availability and data-protection requirements.

12 review controls / 4 high / 6 medium / 2 low

Control 04.01

Priority: High

Choose the right storage type

Choose storage from measured latency, IOPS, throughput, durability, protocol, access mode, and cost requirements. Azure Disk normally provides single-node read/write access; Azure Files provides shared file access. Validate filesystem semantics and driver/OS support, especially for Blob CSI mounts, rather than treating object storage as a fully POSIX-compatible filesystem.

DOCUMENTATION

- Storage considerations for Azure Kubernetes Service
- Choose the right storage type

Control 04.02

Priority: Medium

Size the nodes for storage needs

Size nodes for disk attachment limits, aggregate IOPS and throughput, network bandwidth, and local ephemeral capacity as well as CPU and memory. Include failover, scale-out, and upgrade surge demands; a disk's advertised performance can be constrained by the VM or storage account.

Different types and sizes of nodes are available. Each node (underlying VM) size provides a different amount of core resources such as CPU and memory. These VM sizes have a maximum number of disks that can be attached. Storage performance also varies between VM sizes for the maximum local and attached disk IOPS (input/output operations per second). If your applications require Azure Disks as their storage solution, plan for and choose an appropriate node VM size. The amount of CPU and memory isn't the only factor when you choose a VM size. The storage capabilities are also important.

DOCUMENTATION

- Size the nodes for storage needs

Control 04.03

Priority: Low

Dynamically provision volumes

Prefer dynamic provisioning through explicit CSI StorageClasses and PVCs, while retaining static provisioning for existing volumes or controlled recovery when appropriate. Set reclaimPolicy deliberately: Delete can remove underlying storage after PVC deletion, whereas Retain requires manual lifecycle management. Deleting a pod alone does not normally delete its PVC or volume.

DOCUMENTATION

- Dynamically provision volumes

Control 04.04

Priority: Medium

Secure and back up your data

Encrypt data at rest and in transit where supported, restrict storage access, and use Azure Backup for AKS or another supported solution appropriate to each volume type. Azure Site Recovery is not an AKS cluster backup mechanism. Monitor backup failures and test restores of data and required Kubernetes resources.

Snapshots are generally crash-consistent, not automatically application-consistent or atomic across volumes. Use supported application hooks or database-native backup and recovery to meet consistency needs. Inventory skipped or unsupported volumes, protect external data services separately, and do not install the AKS Backup extension alongside Velero.

RELATED CONTROLS

- Select and test a supported recovery solution for cluster resources and data

DOCUMENTATION

- Secure and back up your data

Control 04.05

Priority: High

Evaluate managed data services for durable application state

Where appropriate, use managed data services such as Azure SQL, Azure Storage, or Azure Cosmos DB instead of operating stateful infrastructure in AKS. Evaluate consistency, latency, connectivity, cost, data residency, backup, and regional failover requirements before choosing.

Externalizing state reduces cluster recovery coupling but does not automatically configure replication or protect against deletion and corruption. Set and test each service's backup and failover policies. Stateful workloads can also run on AKS when their storage and operational requirements are explicitly managed.

DOCUMENTATION

- AKS microservices architecture and external data services

Control 04.06

Priority: Medium

Evaluate and select a suitable node OS disk type

> **Applicability:** Node pools where the VM SKU supports ephemeral OS disks and local placement, capacity, and lifecycle requirements are compatible; managed OS disks remain valid when they better meet requirements.

Compare supported ephemeral and managed OS disks against measured OS, container image, log, and local workload capacity, I/O, provisioning-time, and cost requirements. Ephemeral OS disks can reduce OS disk latency and node provisioning time, but their contents are not durable. Record the selected disk type and test evidence instead of requiring every pool to use ephemeral disks.

Verify SKU placement and capacity requirements and explicitly plan OS disk sizing at node-pool creation. Check compatibility with planned stop/start and scale-down behavior before choosing a disk type. Keep durable application data on suitable persistent storage or managed services; node replacement, reimage, or failure can lose local contents.

DOCUMENTATION

- Ephemeral OS disks in AKS

Control 04.07

Priority: Low

Evaluate Ultra Disk or Premium SSD v2 for demanding storage workloads

> **Applicability:** Only workloads whose measured storage performance requirements justify evaluating Ultra Disk or Premium SSD v2 on supported VM sizes and regions.

Benchmark Ultra Disk and Premium SSD v2 when standard disk tiers cannot meet latency, IOPS, or throughput requirements. Record achieved performance and total cost under representative load; retaining another supported tier is a valid outcome. Validate AKS CSI, VM size, region/zone, caching, encryption, and backup compatibility before selecting a tier.

Provision only justified performance and verify both disk and VM limits. These disk types have availability and configuration restrictions; do not assume ZRS support or that snapshot, restore, and performance changes complete instantly.

DOCUMENTATION

- Use Azure ultra disks on AKS

Control 04.08

Priority: Medium

Keep durable data off container writable layers and node-local storage

Persist data that must survive container, pod, or node replacement on suitable persistent volumes or external services. Treat writable container layers, emptyDir, hostPath, and local NVMe storage as disposable unless application-level replication and recovery explicitly cover their loss.

emptyDir survives a container restart within a pod but is deleted with the pod. A PersistentVolume object alone does not establish durability: local volumes remain tied to a node. Define recovery behavior for caches, checkpoints, and write-ahead logs and test abrupt node loss.

DOCUMENTATION

- Kubernetes volumes

Control 04.09

Priority: Medium

Align disk redundancy, volume binding, and pod topology

For zonal LRS disks, use WaitForFirstConsumer so provisioning respects pod scheduling topology and ensure sufficient nodes can run the pod in the disk's zone. Consider supported ZRS disk SKUs when cross-zone attachment is required. Validate the actual StorageClass and region capabilities rather than assuming defaults; ZRS does not provide simultaneous multi-node writes or automatic application failover.

DOCUMENTATION

- Azure Disk CSI topology and ZRS support
- StorageClass volume binding mode

Control 04.10

Priority: High

Size node OS disks and bound local ephemeral-storage consumption

Size node OS disks from measured image cache, writable layers, logs, pod density, and I/O needs rather than assuming a larger disk alone fixes performance. Set container ephemeral-storage requests and limits, bound emptyDir volumes, configure supported log rotation, and alert on disk pressure and inode exhaustion. OS disk size changes require a replacement node pool.

DOCUMENTATION

- Default OS disk sizing in AKS
- Local ephemeral storage resource management

Control 04.11

Priority: High

Use supported CSI drivers and migrate legacy in-tree volumes

Use supported AKS CSI drivers for Azure Disk, Files, or Blob storage. Inventory legacy in-tree StorageClasses and PVs and follow the documented migration process, including backups and restore tests. Automatic CSI migration does not make an in-tree PV definition eligible for Azure Backup for AKS; verify explicit CSI volume definitions and supported snapshot configuration.

DOCUMENTATION

- Use CSI storage drivers on AKS
- Migrate in-tree volumes to CSI

Control 04.12

Priority: Medium

Test volume expansion, retention, and recovery lifecycle

Alert on persistent-volume capacity and performance, enable allowVolumeExpansion where supported, and test the driver's filesystem expansion and restart requirements. Do not assume volumes can shrink. Review PVC, PV, snapshot, and StorageClass retention together, and test restore and reattachment without deleting the only durable copy.

DOCUMENTATION

- Expanding persistent volume claims

ARCHITECTURE DOMAIN 05 / 11

Networking

Design predictable connectivity, ingress, egress and address space across the application platform.

25 review controls / 8 high / 15 medium / 2 low

Control 05.01

Priority: Medium

Choose Azure CNI IPAM and a supported data plane; migrate kubenet

Prefer Azure CNI Overlay for most workloads to conserve VNet addresses. Use Azure CNI Pod Subnet when pods need directly routable VNet addresses. Choose the IP address management model separately from the data plane, and consider Azure CNI Powered by Cilium for supported Linux workloads.

Overlay pods use a separate pod CIDR, and traffic leaving the cluster is translated to node addresses; they are not directly reachable from connected networks. Flat networking uses VNet pod addresses and needs larger subnets. Kubenet retires on March 31, 2028: inventory existing clusters and validate the supported migration to Azure CNI Overlay, including policies, IP ranges, node reimaging and workload disruption, before that date.

DOCUMENTATION

- CNI networking models and data planes
- Upgrade IPAM and the network data plane
- Azure CNI Overlay
- Bring your own CNI
- Add a node pool with a unique subnet

Control 05.02

Priority: High

Size node subnets and pod address ranges for scale and upgrades

Plan for maximum autoscaler capacity, upgrade surge across node pools, internal load balancer frontends and Azure-reserved addresses. Size pod address space according to the selected Azure CNI IPAM mode, not a single formula for all CNI configurations.

Azure CNI Overlay allocates a /24 pod block per node, even when maxPods is smaller. Flat Node Subnet networking reserves pod addresses based on maxPods; Pod Subnet dynamic and static block allocation have different reservation rules. Include simultaneous upgrades and future pools, and monitor address utilization.

DOCUMENTATION

- Plan IP addressing for your cluster

Control 05.03

Priority: Medium

Use supported HTTP/HTTPS ingress and migrate retired ingress-nginx

> **Applicability:** Applies to workloads using HTTP/HTTPS ingress, including deployments migrating existing ingress-nginx or managed NGINX controllers.

Use a supported Gateway API implementation or ingress controller for shared HTTP/HTTPS routing and TLS; LoadBalancer services remain appropriate for L4 traffic. Upstream ingress-nginx maintenance ended in March 2026. Microsoft's critical security patch support for application routing add-on NGINX ends in November 2026: migrate before support ends.

Evaluate the application routing Gateway API implementation, Application Gateway for Containers, or Gateway API ingress with the Istio add-on against feature and regional requirements. Inventory NGINX annotations, snippets, DNS, certificates and private frontend settings; test equivalents and staged cutover. The application routing Gateway API implementation does not support all NGINX capabilities and cannot coexist with the Istio service mesh add-on.

DOCUMENTATION

- Application routing Gateway API implementation
- Managed NGINX support deadline and migration options
- Application Gateway for Containers

Control 05.04

Priority: Medium

Secure your exposed applications with a web application firewall (WAF)

> **Applicability:** Applies to internet-facing HTTP/HTTPS applications whose threat model requires a web application firewall.

For internet-facing HTTP/HTTPS applications, evaluate Azure Application Gateway WAF or Azure Front Door WAF. Enable appropriate managed and custom rules, tune false positives, move to prevention after validation, and monitor alerts. A WAF protects supported web protocols, not arbitrary TCP/UDP traffic; prevent clients from bypassing it by restricting direct access to origins.

DOCUMENTATION

- Secure traffic with a web application firewall (WAF)

Control 05.05

Priority: Low

Restrict ingress and Gateway API hostnames and cross-namespace references

Authorize which teams can create or change Ingress, Gateway and Route resources. Enforce approved hostnames and controller classes through admission policy. Scope Gateway listeners and allowedRoutes, and approve cross-namespace ReferenceGrants explicitly to prevent route or certificate hijacking.

DOCUMENTATION

- Application routing Gateway API configuration

Control 05.06

Priority: High

Use internal load balancers for private applications

Choose public or internal frontends deliberately. Use an internal load balancer for private applications and restrict access through connected networks. For internet-facing applications, expose only required ports and route through the chosen edge/WAF design. Configure private DNS and validate the controller's internal frontend settings.

DOCUMENTATION

- Use an internal load balancer with AKS

Control 05.07

Priority: Medium

Control traffic flow with network policies

Enforce intended pod communication boundaries with deployed network policies and a compatible enforcement engine. Verify required connections succeed and prohibited connections fail from representative pods and namespaces; an engine setting alone is not proof of isolation.

Prefer Azure CNI Powered by Cilium for supported Linux configurations. For existing Azure NPM Linux deployments, plan and test migration before its September 30, 2028 retirement. Retain the effective policy inventory, pod/namespace selector coverage and repeatable allowed/denied connectivity tests, including after upgrades and policy changes. Network policies govern pod traffic while NSGs protect subnet boundaries; enabling an engine without applicable policies does not restrict traffic.

DOCUMENTATION

- Network policy best practices
- Secure traffic between pods using network policies

TOOLS

- Calico
- Cilium

Control 05.08

Priority: Medium

Configure default network policies in each namespace

Apply default-deny ingress and egress policies to application namespaces, then allow required flows explicitly. Include DNS, identity/token endpoints, monitoring, ingress health checks and application dependencies. Validate policies with the actual data plane; do not blindly deny system namespace traffic.

DOCUMENTATION

- Recipes of best default network policies
- AKS network policy engines and limitations

Control 05.09

Priority: Medium

Filter egress traffic with AzFW/NVA if your security requirements mandate it (egress lockdown)

> **Applicability:** Applies when workload or organizational requirements mandate inspected or allowlisted outbound traffic.

Where egress inspection is required, use user-defined routing with Azure Firewall or an approved NVA and allow the documented AKS platform dependencies as well as workload destinations. Validate DNS, private endpoints, identity and upgrade traffic. A UDR alone does not prove that egress is filtered; configure and monitor firewall rules and prevent bypass paths.

Retain effective routes, firewall policy/rule ownership and enforcement logs. Test required destinations and explicitly prohibited destinations from representative node pools and pods, including alternative public/private paths and behavior during firewall failure. Revalidate platform/bootstrap and upgrade dependencies after routing or policy changes.

DOCUMENTATION

- Egress traffic requirements

Control 05.10

Priority: Medium

Restrict pod access to node IMDS and managed identity tokens

Restrict application pod egress to 169.254.169.254 using policies supported by the selected data plane, and verify enforcement. Use Microsoft Entra Workload ID rather than node identities for Azure access. Keep node identity permissions minimal and restrict privileged and hostNetwork pods, which can bypass ordinary pod isolation.

AKS IMDS restriction is documented as preview and blocks non-host-network pods only. Evaluate its add-on compatibility and preview support limitations before enabling it; it is not a universal production substitute for least-privilege identities and workload isolation.

DOCUMENTATION

- Restrict access to the Instance Metadata API
- AKS IMDS restriction (preview)

Control 05.11

Priority: High

Use private clusters if your requirements mandate it

> **Applicability:** Applies when requirements mandate a private Kubernetes API endpoint or the cluster already uses private API connectivity.

In a private cluster, the control plane or API server has internal IP addresses and is not exposed to Internet

Plan private DNS resolution, VPN/ExpressRoute or peered-network connectivity, deployment agents and emergency administration before disabling public API access. API Server VNet Integration supports public and private clusters and is not itself proof that public access is disabled. Check regional availability (currently excluding Qatar Central in the public cloud) and subnet requirements. Govern AKS Run Command permissions because it can provide management access without direct private network connectivity.

DOCUMENTATION

- Create a private cluster (using private link)
- API Server VNet Integration and availability
- Manage Kubernetes resources and Run Command through the portal
- Private cluster public FQDN and DNS considerations

Control 05.12

Priority: Medium

Choose global traffic routing for multi-region applications

> **Applicability:** Applies to multi-region applications requiring global traffic distribution or regional failover.

For multi-region applications, choose Azure Front Door for global HTTP/HTTPS proxying and edge/WAF capabilities, or Azure Traffic Manager for DNS-based routing across supported protocols. Global routing is not necessary for every single-region cluster.

Configure application-aware health probes and regional origin priorities; measure failover and recovery against RTO under realistic traffic. Traffic Manager answers DNS queries rather than proxying application traffic, so test DNS TTLs and client caching. Front Door provides an HTTP/HTTPS data path; test TLS, private-origin connectivity and connection draining for the chosen design. Consider supported cross-region L4 routing separately when required by protocol. Align routing with data replication and regional capacity: traffic failover neither replicates data nor eliminates detection and recovery delays.

DOCUMENTATION

- Multi-region AKS best practices
- Global load balancing for AKS
- Cross-region Layer 4 load balancing

Control 05.13

Priority: High

Set maxPods explicitly for the selected IPAM mode and workload density

Do not assume a universal default of 30 pods per node: defaults and limits depend on networking mode and configuration. Choose maxPods using application density, system DaemonSets, node resources and IP capacity.

Overlay pod addresses are not VNet-routable, while flat networking allocates VNet addresses. Verify documented limits for the chosen IPAM mode and OS. Changing maxPods can require a replacement node pool; validate it before production deployment.

DOCUMENTATION

- Configure Azure CNI networking on AKS

Control 05.14

Priority: Medium

If using a public API endpoint, restrict the IP addresses that can access it

> **Applicability:** Applies when the Kubernetes API server has a public endpoint.

To improve the security of your clusters and minimize the risk of attacks, we recommend limiting the IP address ranges that can access the API server.

DOCUMENTATION

- Secure access to the API server using authorized IP address ranges

Control 05.15

Priority: Medium

Choose an explicit outbound method and size SNAT capacity

Choose load balancer, managed/user-assigned NAT Gateway, or user-defined routing according to egress scale and inspection needs; evaluate network-isolated mode when cluster bootstrapping must not depend on public egress. NAT Gateway provides scalable SNAT and predictable public egress IPs, but is not a firewall. Monitor connection and SNAT utilization, plan public IP capacity and zone resilience, and test IP allowlists when changing outboundType.

DOCUMENTATION

- Managed NAT Gateway

Control 05.16

Priority: Medium

Use Azure CNI Pod Subnet allocation when VNet-routable pods are required

> **Applicability:** Applies when pods require directly routable VNet addresses or an existing deployment uses Azure CNI Pod Subnet.

Prefer Overlay when directly routable pod addresses are unnecessary. For flat networking, separate pod and node subnets with Azure CNI Pod Subnet and choose dynamic or static block allocation based on supported scale and workload requirements.

Dynamic allocation allocates pod IPs on demand in batches; it reduces preallocation but does not eliminate subnet exhaustion. Static block allocation has different capacity and migration constraints. Monitor available addresses and plan pool/subnet migration rather than assuming a full cluster rebuild is always required.

DOCUMENTATION

- Azure CNI Pod Subnet networking

Control 05.17

Priority: Medium

Use Private Endpoints (preferred) or Virtual Network Service Endpoints to access PaaS services from the cluster

Choose Private Endpoints where private IP connectivity and disabling public service access are required. Service Endpoints instead secure access to a service's public endpoint from selected subnets; they are not equivalent to Private Link. Configure service-side access controls and private DNS, and validate workload and deployment-agent resolution.

DOCUMENTATION

- What is Azure Private Link?

Control 05.18

Priority: Medium

Consider using a service mesh for advanced microservice communication management

> **Applicability:** Applies when a service mesh is installed or documented workload mTLS, identity or traffic-management requirements justify one; adoption is not required.

Use a service mesh only when requirements justify its resource and operational overhead, for example workload mTLS, service identity or advanced traffic management. Evaluate the supported Istio add-on, compatibility, revision upgrades and explicit authorization policies; installing a mesh alone does not enforce zero trust. The managed Open Service Mesh add-on ends support on September 30, 2027: migrate affected clusters to Istio before then.

DOCUMENTATION

- About service meshes

Control 05.19

Priority: Low

If required add your own CNI plugin

> **Applicability:** Applies to existing BYO CNI deployments or documented requirements that supported Azure CNI options cannot satisfy.

Use bring-your-own CNI only when supported Azure CNI options cannot meet requirements. Review AKS feature compatibility and support boundaries: installation, configuration, upgrades and troubleshooting of the custom CNI are your responsibility. Validate networking before deploying workloads.

DOCUMENTATION

- Bring your own Container Network Interface (CNI) plugin for AKS

Control 05.20

Priority: Medium

If using Azure CNI, consider using different Subnets for NodePools

> **Applicability:** Applies when existing node pools use separate subnets or address-space and network-boundary requirements justify distinct node-pool subnets.

This can address requirements such as having non-contiguous virtual network address space to split across node pools and other security reasons.

DOCUMENTATION

- Add a node pool with a unique subnet

Control 05.21

Priority: Medium

Verify non overlapping CIDR ranges for Service-CIDR and Pod-CIDR or Subnet-CIDR

Ensure service, pod and node subnet ranges do not overlap one another or connected VNet, peered, VPN and on-premises networks. Reserve DNS service addresses correctly and account for future connectivity. Overlay address overlap with external destinations can prevent pods from reaching those destinations.

DOCUMENTATION

- Pod CIDR, Service CIDR and Subnet CIDR
- Use the right CIDR ranges for networking parameters

Control 05.22

Priority: High

Validate resilient DNS resolution for cluster and private dependencies

Ensure nodes and pods can resolve Kubernetes services, the API server, private endpoints and external dependencies using the intended DNS path. Link private DNS zones to the correct VNets and configure conditional forwarding or Azure DNS Private Resolver for connected networks.

Allow both UDP and TCP port 53 through NSGs, firewalls and network policies; test large responses and failover. Monitor DNS latency and failures, scale CoreDNS appropriately and use supported ConfigMap customization rather than editing the managed Corefile. Evaluate LocalDNS only after checking current OS, Kubernetes and networking compatibility.

DOCUMENTATION

- Private cluster DNS requirements
- Customize CoreDNS on AKS

Control 05.23

Priority: High

Use Standard Load Balancer and migrate unsupported Basic deployments

> **Applicability:** Applies to clusters using Azure Load Balancer, including remaining unsupported Basic deployments.

AKS stopped supporting Basic Load Balancer on September 30, 2025. Use Standard for new clusters and urgently migrate remaining Basic deployments through the documented AKS upgrade or a replacement cluster.

The documented upgrade causes downtime and cannot be rolled back. Review Kubernetes/CLI prerequisites, admission webhooks and private Key Vault KMS dependencies, then validate services, health probes and new outbound public IP allowlists. Availability Set clusters have a separate combined migration path.

DOCUMENTATION

- Upgrade Basic Load Balancer on AKS

Control 05.24

Priority: High

Evaluate network-isolated bootstrap and explicitly permit required dependencies

> **Applicability:** Applies when network-isolated AKS is required or already configured; this mode is not required for every production cluster.

For workloads requiring network-isolated AKS, configure a private API path and bootstrap-artifact-source Cache with supported AKS-managed or bring-your-own ACR connectivity. Outbound type none is generally available but does not itself block every possible egress path; block is preview and has different network support. Validate private DNS, registry permissions and application access before removing outbound connectivity.

Windows pools and kubenet aren't supported. Use supported node image upgrade channels rather than assuming SecurityPatch is available. Review every add-on's outbound dependencies and support matrix; Dapr, Azure Machine Learning and Flux extensions are not currently supported in this mode. For existing clusters, follow the documented cache transition and node reimage sequence while preserving required outbound access until reimaging completes.

DOCUMENTATION

- Network-isolated AKS concepts, limitations and add-on dependencies
- Create or migrate a network-isolated AKS cluster

Control 05.25

Priority: High

Validate application TLS issuance, rotation and end-to-end trust

> **Applicability:** Applies to application TLS endpoints, including Gateway/ingress listeners, private application endpoints and encrypted backend connections.

Own application certificate issuance, renewal and revocation, with expiry and failed-renewal alerts. Use a supported controller and secrets integration, and verify that a renewed certificate reaches the listener and is served without an availability regression. Certificate storage or secret rotation alone does not demonstrate listener reload.

For supported application routing Gateway API integration, use an unversioned Key Vault certificate URI, a correctly federated service account and least-privilege Key Vault access; use the documented integration for the chosen controller rather than legacy NGINX flags. Test issuance, rotation propagation and listener reload, SAN/hostname matching, certificate chain and client trust, plus backend certificate validation wherever TLS is re-encrypted. Retain externally observed TLS handshake and renewal evidence for the entire intended encrypted path. Application certificate lifecycle is separate from AKS node and cluster CA rotation.

DOCUMENTATION

- Configure Azure DNS and TLS with the application routing Gateway API implementation

ARCHITECTURE DOMAIN 06 / 11

Resource Management

Balance workload demand, scheduling, scaling and cost without sacrificing reliability.

15 review controls / 4 high / 5 medium / 6 low

Control 06.01 Priority: Low

Evaluate virtual nodes only for ACI-compatible burst workloads

> **Applicability:** Only Linux burst workloads compatible with Azure Container Instances and the selected virtual-node implementation's networking, storage, identity, and Kubernetes API limitations.

Virtual nodes can place compatible Linux pods on Azure Container Instances for rapid burst capacity. Record a representative burst test comparing startup latency, workload compatibility, regional capacity, and total cost with Cluster Autoscaler or node auto-provisioning. Choosing regular nodes is a valid outcome; adoption alone is not evidence of improvement.

Validate the feature matrix for the selected implementation: Virtual Nodes v2 is a preview extension, not the original add-on, and its preview support terms must be acceptable. V2 supports init containers and Azure Files PVCs but requires Azure CNI node-subnet mode and doesn't support Overlay, Cilium, NPM, Windows, BYO VNets, or daemonsets. Account for extension CPU and memory overhead and keep system components on regular AKS nodes.

RELATED CONTROLS

- Enable cluster autoscaling
- Constrain node auto-provisioning without making pods unschedulable

DOCUMENTATION

- Burst to Azure Container Instances
- Virtual Nodes v2 feature matrix (preview)

Control 06.02

Priority: Medium

Size nodes against allocatable resources and workload failure domains

Measure workload requests, peaks, and sustained consumption before selecting node sizes and counts. Use allocatable resources after Kubernetes reservations and daemonset overhead, not raw SKU capacity. Include pod density, disk attachment and throughput limits, network bandwidth, architecture, and VM-family quotas.

Large nodes can improve bin packing but increase the failure blast radius or drain impact; small nodes increase per-node overhead and can leave large pods unschedulable. Record capacity tests showing that large pods fit and enough headroom remains for failover and surge upgrades. Where supported, node auto-provisioning can select VM sizes dynamically, but still depends on accurate pod requests and capacity constraints.

RELATED CONTROLS

- Profile container requests and memory limits; evaluate CPU limits contextually
- Coordinate pod autoscaling with node capacity and resource requests

DOCUMENTATION

- AKS node resource reservations
- Choose the right storage type
- AKS virtual machine sizes and restrictions

TOOLS

- Kubernetes instance calculator

Control 06.03

Priority: Medium

Benchmark Arm64 nodes for compatible Linux workloads

> **Applicability:** Only compatible Linux workloads for which Arm64 VM availability and the application, agent, and security support matrix permit an architecture choice.

Evaluate Arm64 price-performance using representative workloads rather than assuming universal savings. Record throughput, latency, resource use, and cost against the existing architecture; retaining x86 is a valid outcome. Build multi-architecture images and verify application binaries, agents, daemonsets, security integrations, and regional VM availability before adding an Arm64 pool.

Arm64 isn't supported for Windows node pools. Existing x86 pools can't be converted to Arm64; create a new pool and migrate compatible workloads with architecture-aware scheduling.

DOCUMENTATION

- Add an ARM64 node pool

Control 06.04

Priority: Low

Enforce resource quotas for namespaces

Apply namespace ResourceQuotas appropriate to tenant needs, covering aggregate requests or limits, storage, and object counts. Pair them with reviewed LimitRange defaults or admission policy, and monitor quota-related pod creation failures.

The scheduler places pods using requests, not limits. Quotas are admission controls, not guaranteed capacity: a Deployment can be accepted while its pods are rejected for exceeding quota. Stage enforcement for existing tenants and allow headroom for rolling updates and autoscaling.

DOCUMENTATION

- Enforce resource quotas
- Resources quotas

Control 06.05

Priority: Low

Set workload-appropriate LimitRange defaults and bounds

Use LimitRange to default or bound requests and limits within a namespace where this matches tenant policy. Defaults aren't a substitute for application profiling; overly small defaults cause throttling or OOM failures, while excessive requests waste capacity and distort autoscaling.

DOCUMENTATION

- LimitRange

Control 06.06

Priority: High

Profile container requests and memory limits; evaluate CPU limits contextually

Set measured CPU and memory requests for application containers, sidecars, and init containers, and choose memory limits with adequate peak headroom. Bound local ephemeral storage with suitable requests, limits, and emptyDir size limits. Requests drive scheduling and utilization-based HPA; limits don't prevent a memory leak or provide reserved capacity.

CPU limits throttle execution and can harm latency-sensitive workloads even when the node has spare CPU; evaluate them against tenant isolation requirements. Exceeding memory limits can trigger OOM termination, with restart behavior determined by the workload and restart policy, not guaranteed pod relocation. Review CPU throttling, OOM events, memory peaks, and ephemeral-storage pressure from production telemetry or VPA recommendations, and confirm changes with representative load tests and the intended Quality of Service behavior.

RELATED CONTROLS

- Size node OS disks and bound local ephemeral-storage consumption
- Coordinate pod autoscaling with node capacity and resource requests

DOCUMENTATION

- Define pod resource requests and limits
- Assign Memory Resources to container
- Managing resources for containers
- Configure Quality of Service for pods

Control 06.07

Priority: Low

Allocate shared cluster costs with AKS cost analysis or an equivalent

Evaluate the OpenCost-based AKS cost analysis add-on for namespace and cluster cost attribution in Azure Cost Management. It requires Standard or Premium tier, managed identity, the Azure Disk CSI driver, and a supported billing offer; Kubernetes cost views support Enterprise Agreement and Microsoft Customer Agreement offers.

The add-on consumes cluster resources, doesn't support virtual nodes, and has documented scale and regional limits. Use Kubecost or another solution when required allocation granularity or integrations aren't covered. Establish ownership labels and an explicit policy for idle and shared platform costs before implementing chargeback.

DOCUMENTATION

- AKS cost analysis

Control 06.08

Priority: Medium

Evaluate multi-instance GPU partitioning for compatible workloads

> **Applicability:** Only Linux GPU workloads using supported MIG-capable NVIDIA VM sizes where sharing a GPU meets application memory, throughput, and isolation requirements.

Supported NVIDIA A100, H100, and H200 VM sizes can use multi-instance GPU (MIG) partitioning to share a GPU as isolated compute and memory slices. Choose a supported profile based on workload memory and throughput; smaller slices improve utilization only when the application fits. Record representative concurrency, throughput, memory use, and total cost against an unpartitioned GPU rather than treating adoption as the outcome.

MIG isn't supported for Windows node pools. The GPU instance profile can't be changed after pool creation, so plan migration to another pool if requirements change. Validate device plugin configuration, scheduling resource names, driver compatibility, and regional GPU quota.

DOCUMENTATION

- Multi-instance GPU Node pool

Control 06.09

Priority: Medium

Stop eligible idle user node pools when interruption is acceptable

> **Applicability:** Only eligible development, test, or batch user node pools whose workloads can tolerate downtime or move elsewhere; not system pools or clusters using node auto-provisioning.

Schedule stop/start for development or batch user pools only when workloads can tolerate downtime or move elsewhere. System pools can't be stopped individually, and node pool stop/start isn't supported on clusters with node auto-provisioning.

Stopping a pool also stops its Cluster Autoscaler until restart. Record restart and workload recovery times against the allowed downtime and verify net savings including disks, networking resources, and other services that can still incur charges. Don't modify the underlying VM scale set while stopped.

DOCUMENTATION

- Start and stop an Azure Kubernetes Service (AKS) node pool

Control 06.10

Priority: Low

Use node pool snapshots for repeatable supported configurations, not data backup

> **Applicability:** Only workflows that need to reproduce supported node pool configurations and images across pools or clusters; not workload-data backup or recovery.

Node pool snapshots capture configuration and image information for creating consistent pools or clusters while those versions remain supported. Test pool creation from the snapshot and verify the resulting configuration, supported versions, and workload readiness. Snapshots don't back up workload state, persistent volumes, or Kubernetes objects, and don't extend Kubernetes or OS support lifecycles.

DOCUMENTATION

- AKS node pool snapshot

Control 06.11

Priority: High

Ensure your subscription has enough quota to scale out your nodepools

Budget regional total and VM-family vCPU quotas for maximum pool sizes, autoscaling, upgrade surge, and failover. Also check subnet IP capacity, disk and network limits, and regional SKU availability. Approved quota doesn't reserve physical capacity, so test fallback VM sizes or regions where the workload permits.

DOCUMENTATION

- Azure subscription and service limits, quotas, and constraints

Control 06.12

Priority: Low

Choose Delete or Deallocate scale-down mode based on startup and storage costs

> **Applicability:** Only supported node pools with planned scale-down activity; evaluate Deallocate only for compatible managed OS disks and workloads that benefit from retained images.

Delete is the default scale-down behavior. Deallocate retains managed OS disks and cached images for potentially faster restarts, while VM compute billing stops; storage charges continue. Deallocate doesn't support ephemeral OS disks.

Compare measured restart time and total retained-disk cost before choosing a mode; retaining the default Delete mode is a valid outcome. Deallocated nodes can remain registered as NotReady. Changing the mode to Delete removes previously deallocated nodes, so don't treat them as durable capacity or a backup. Use the supported AKS scale-down APIs rather than directly stopping VMs.

DOCUMENTATION

- Use Scale-down Mode to delete/deallocate nodes

Control 06.13

Priority: High

Coordinate pod autoscaling with node capacity and resource requests

> **Applicability:** Workloads with variable demand that require horizontal or vertical autoscaling, and clusters whose node capacity must respond to that demand.

Use HPA for workload demand or KEDA for event-driven scaling, together with Cluster Autoscaler or node auto-provisioning for capacity. Validate metrics availability, minimum replicas, startup time, cooldowns, and downstream service capacity under load.

HPA CPU utilization depends on CPU requests. Review observed CPU, memory, throttling, OOM events, and ephemeral-storage consumption when rightsizing. Start VPA in recommendation mode and account for update-mode disruption before enabling automatic changes. Don't combine automatic VPA changes with HPA scaling on the same CPU or memory metrics; use recommendations or deliberately separate signals such as custom/external metrics. Load-test scale-out and scale-in against latency, availability, queue-drain, and recovery targets, including namespace quotas, node provisioning delays, and resource exhaustion. Scaling pods alone doesn't create nodes, and scaling nodes doesn't fix quota, topology, or admission failures.

RELATED CONTROLS

- Implement autoscaling of your applications (Horizontal pod autoscaler, KEDA)
- Enable cluster autoscaling
- Constrain node auto-provisioning without making pods unschedulable

DOCUMENTATION

- AKS scaling overview
- Vertical Pod Autoscaler

Control 06.14

Priority: High

Constrain node auto-provisioning without making pods unschedulable

> **Applicability:** Only clusters using or evaluating node auto-provisioning, including AKS Automatic; not a requirement to replace a suitable Cluster Autoscaler design.

For NAP, define Karpenter NodePool and AKSNodeClass constraints, aggregate resource limits, allowed VM families, zones, and capacity types according to workload requirements. Keep enough SKU flexibility for allocation failures and review consolidation and disruption budgets.

NAP-managed nodes follow the control plane Kubernetes version. Windows pools and IPv6 clusters aren't supported, and NAP prevents cluster and node pool stop/start. AKS Automatic preconfigures NAP; evaluate Standard when custom platform requirements demand more control. Avoid having multiple autoscalers manage the same capacity.

DOCUMENTATION

- Node auto-provisioning overview and limitations
- Configure NAP node pools
- Configure NAP disruption policies

Control 06.15

Priority: Medium

Right-size before purchasing commitments and monitor total platform cost

Apply reservations or Azure savings plans to measured steady-state usage only after rightsizing and autoscaling. Compare commitment scope and flexibility with expected changes in VM families, regions, architecture, and workload demand; commitments don't guarantee physical capacity.

Track cluster management, compute, disks, load balancers, public IPs, egress, observability, and security services, not only node prices. Set budgets and anomaly alerts with owners, review commitment utilization, and avoid committing to capacity intended to be stopped or replaced by interruptible Spot workloads. Budget alerts notify; they aren't automatic spending caps.

DOCUMENTATION

- AKS cost optimization best practices

ARCHITECTURE DOMAIN 07 / 11

Cluster Operations

Make cluster changes observable, repeatable and recoverable throughout the platform lifecycle.

33 review controls / 14 high / 11 medium / 8 low

Control 07.01

Priority: High

Keep node images current with a tested node OS upgrade policy

AKS releases Linux node images weekly and Windows images monthly; regional rollout can take up to two weeks. Configure node OS automatic upgrades or an equivalent monitored process to apply OS, runtime, and AKS component fixes without necessarily changing the Kubernetes version.

Choose NodeImage for complete image updates, or evaluate SecurityPatch for supported Linux configurations when reducing disruption is important. SecurityPatch isn't supported for Windows. Configure the separate node OS maintenance schedule, test drains and PodDisruptionBudgets, and monitor failed upgrades. AKS Automatic preconfigures NodeImage. Kured only coordinates Linux reboots and isn't a substitute for AKS image lifecycle management.

DOCUMENTATION

- Azure Kubernetes Service (AKS) node image upgrades
- Automatic node OS image upgrades and channel limitations
- Use Event Grid to know when an upgrade is available

TOOLS

- Kured (KUbernetes REboot Daemon)

Control 07.02

Priority: Low

Consider proximity placement groups for latency-sensitive workloads

> **Applicability:** Applies when measured latency requirements justify proximity placement groups or existing node pools already use them.

Use proximity placement groups only when measured inter-node latency requirements justify physical colocation. Retain before/after workload measurements and verify that capacity constraints and shared failure exposure remain acceptable; adopting a placement group is not itself a readiness improvement.

A proximity placement group maps to one availability zone and can constrain capacity allocation; it doesn't change control plane placement or its SLA. If measured latency justifies colocation, use separate zonal user pools and placement groups while keeping system pools and application replicas distributed across zones.

DOCUMENTATION

- Reduce latency with proximity placement groups

Control 07.03

Priority: Low

Own the lifecycle and health of installed cluster extensions

> **Applicability:** Applies when cluster extensions are installed or an approved workload dependency requires an extension.

Inventory installed cluster extensions and assign owners for supported versions, health, permissions, upgrades and removal. Verify compatibility with the cluster's Kubernetes version, OS and networking before rollout; do not install extensions solely to satisfy this check.

Document support status, resource overhead, network dependencies, upgrade policy and rollback or recovery for each extension. Monitor unhealthy installations and reconcile configuration drift. Extension lifecycle support can end independently from Kubernetes support, so include extensions in platform upgrade rehearsals.

DOCUMENTATION

- Cluster extensions for Azure Kubernetes Service

Control 07.04

Priority: Medium

Securely connect to nodes through a bastion host

Avoid public SSH or RDP exposure on AKS nodes. Prefer Kubernetes-native debugging for routine diagnosis; where direct node access is necessary, use a controlled private path such as Azure Bastion with least-privilege, audited, time-limited access.

DOCUMENTATION

- Connect securely to AKS cluster nodes

Control 07.05

Priority: Medium

Regularly assess cluster health and configuration

Combine AKS diagnostics, Azure Advisor, policy compliance, and maintained Kubernetes assessment tools. Before enforcing resource quotas or admission policies, inventory existing workloads and test the policy in audit mode to avoid blocking deployments.

DOCUMENTATION

- Azure Kubernetes Service diagnostics

TOOLS

- AKS Periscope
- kubectl
- kubebench

Control 07.06

Priority: High

Define an observability architecture for logs, metrics, and traces

Monitor control plane, nodes, workloads, and application health. Azure Monitor Container insights collects logs into a Log Analytics workspace; managed Prometheus stores metrics in an Azure Monitor workspace; application instrumentation provides traces. These are complementary signals, not interchangeable stores.

Choose managed or self-managed backends based on operational ownership, compliance, retention, and cost. Verify network access and managed identity authentication, and avoid collecting the same telemetry through multiple agents. AKS Automatic onboarding defaults don't remove the need to verify collection and alerting.

DOCUMENTATION

- Azure Monitor for AKS
- Azure Managed Grafana

TOOLS

- Elastic Cloud
- Datadog

Control 07.07

Priority: High

Collect Kubernetes and application metrics with managed Prometheus or an equivalent

Azure Monitor managed service for Prometheus provides Prometheus-compatible collection and storage without operating a Prometheus server. Enable default Kubernetes scraping and configure application exporters or scrape targets where needed.

Use Grafana dashboards and Prometheus recording and alert rules to track availability, saturation, and workload SLOs. Limit metric cardinality and scrape scope to control cost. The older Container insights Prometheus-to-Log-Analytics integration isn't the same as managed Prometheus.

DOCUMENTATION

- Azure Monitor managed service for Prometheus
- Customize managed Prometheus scraping

Control 07.08

Priority: Low

Configure distributed tracing

Instrument supported application runtimes with OpenTelemetry and export traces to Application Insights or another compatible backend. Propagate trace context across services, correlate traces with logs, and choose sampling and sensitive-data filtering appropriate to the workload.

DOCUMENTATION

- Enable Azure Monitor OpenTelemetry
- Distributed tracing in Application Insights

Control 07.09

Priority: Medium

Operate a controlled Kubernetes upgrade mechanism

Configure patch, stable or rapid according to validation lead time, or operate an equivalent owned and monitored upgrade schedule. Verify that upgrades start, complete and recover from failure; channel configuration alone is not evidence that the cluster remains supported. Patch remains on a minor version, so schedule validated minor-version transitions explicitly.

Kubernetes auto-upgrade and node OS image channels are separate. AKS Automatic preconfigures upgrades; AKS Standard requires an explicit policy. Stage changes in nonproduction and monitor completion. Starting June 1, 2027, AKS documents platform-initiated upgrades after unsupported versions exhaust the 60-day grace period; these can bypass maintenance windows and blocking PDBs.

DOCUMENTATION

- Set AKS auto-upgrade channel

Control 07.10

Priority: Medium

Enable cluster autoscaling

To keep up with application demands in Azure Kubernetes Service (AKS), you may need to adjust the number of nodes that run your workloads. The cluster autoscaler component can watch for pods in your cluster that can't be scheduled because of resource constraints.

For fixed VM-size pools, configure Cluster Autoscaler minimum and maximum counts per pool and a shared cluster autoscaler profile. Alternatively evaluate managed Karpenter-based node auto-provisioning (NAP), which selects capacity from pending pod requirements. AKS Automatic preconfigures NAP; use AKS Standard for supported custom configurations. Validate resource requests, placement constraints, quotas, and disruption settings before expecting either approach to scale successfully.

DOCUMENTATION

- AKS Autoscaler
- Node autoprovisioning (karpenter)
- Cluster autoscaler

Control 07.11

Priority: Medium

Control versioned desired state and configuration drift

Keep approved desired state in version control and verify deployed configuration against it, with owned drift detection, reconciliation and rollback. GitOps is one implementation choice, not a mandatory product or a separate substitute for CI build, test and artifact-promotion gates.

Use a supported Flux extension, a maintained Argo CD deployment, or a controlled pipeline with equivalent reviewable state and drift handling. Microsoft.Flux supports AKS without Azure Arc onboarding. Define reconciliation ownership, secret handling, environment promotion, emergency changes and rollback, and ensure CI/CD and controllers do not continually overwrite the same resources. Retain evidence that unauthorized drift is detected and resolved.

DOCUMENTATION

- Use Flux v2 addon for AKS
- Use ArgoCD with AKS

Control 07.12

Priority: High

Implement CI/CD to deploy workloads in your cluster

Use CI/CD to build, test, scan, and promote immutable container images and manifests. Prefer federated identity and scoped deployment permissions over stored cluster-admin credentials; private clusters also require a deliberate runner network path.

DOCUMENTATION

- Automated deployments for Azure Kubernetes Service
- Build and deploy to Azure Kubernetes Service with Azure Pipelines
- Tutorial: Deploy from GitHub to Azure Kubernetes Service using Jenkins

Control 07.13

Priority: Low

Validate justified node tuning with measurements and rollback

> **Applicability:** Applies when measured workload performance requires nondefault OS or kubelet settings, or existing custom node settings need lifecycle validation.

Tune OS or kubelet settings only for a documented workload need. Capture baseline and post-change performance, saturation and error measurements under realistic load, test in nonproduction first, and define acceptance thresholds and a supported rollback or replacement-pool plan.

Use only supported AKS configuration interfaces and validate settings against the node OS, Kubernetes version and cluster mode. Windows does not support the Linux OS configuration surface. Avoid direct VM edits that can be lost during reimage and fall outside AKS support; version custom settings and revalidate them after image or Kubernetes upgrades.

DOCUMENTATION

- Customize node configuration for Azure Kubernetes Service (AKS) node pools

Control 07.14

Priority: Low

Consider using AKS command invoke on private clusters

You can use command invoke to access private clusters without having to configure a VPN or Express Route

Command invoke runs kubectl and helm through the Azure API and can bypass the usual private network access path. Restrict its Azure RBAC permissions and audit use; disable it when policy requires all access through private connectivity. It has execution and pod-scheduling constraints and isn't a replacement for a reliable deployment runner or emergency access design.

DOCUMENTATION

- Use command invoke to access a private AKS

Control 07.15

Priority: Medium

Monitor CPU and memory utilization of the nodes

Monitor allocatable versus requested and consumed CPU and memory, CPU throttling, memory pressure, OOM kills, pending pods, and disk pressure. Use sustained trends and application latency rather than average node utilization alone when rightsizing.

DOCUMENTATION

- Monitor your Kubernetes cluster performance

Control 07.16

Priority: Medium

Regularly check Azure Advisor for recommendations on your cluster

Review Azure Advisor recommendations for reliability, security, performance, operational excellence, and cost. Assign owners and validate recommendations against workload SLOs and measured demand rather than applying every recommendation automatically.

DOCUMENTATION

- Azure advisor get started

Control 07.17

Priority: Low

Verify certificate rotation and rehearse manual CA rotation

Node client and serving certificate autorotation is enabled by default on RBAC-enabled clusters. Cluster CA rotation is a separate manual operation that can disrupt access and requires refreshed kubeconfigs. Inventory certificate dependencies and test the recovery procedure instead of assuming every certificate rotates automatically.

The opt-out tag for kubelet serving certificate rotation stops being honored on April 1, 2027. Update agents and workloads to trust the correct certificate path before their next reimage; manage application ingress certificates independently.

DOCUMENTATION

- Certificate rotation in Azure Kubernetes Service (AKS)

Control 07.18

Priority: High

Store and analyze your cluster logs with Container Insights (or other tools like Telegraf/ElasticSearch)

Collect application stdout/stderr and relevant Kubernetes events with Container insights or an equivalent backend. Define namespace and container exclusions, retention, access controls, and sensitive-data handling; verify ingestion and queries during incidents.

DOCUMENTATION

- Container insights overview

Control 07.19

Priority: Medium

Migrate legacy ContainerLog collection and queries to ContainerLogV2

> **Applicability:** Applies to deployments that still collect legacy ContainerLog data or have queries, alerts, workbooks or exports using its schema.

Support for the legacy ContainerLog table retired on September 30, 2026. Use ContainerLogV2 and update alert rules, workbooks, exports, and saved queries to its schema. Any temporary acceptance of legacy data after retirement isn't a supported migration strategy.

ContainerLogV2 includes pod and namespace metadata and supports multiline logging and Basic Logs. Evaluate query and alert limitations before choosing a cheaper table plan; verify DCR and ConfigMap settings because the ConfigMap takes precedence.

DOCUMENTATION

- Enable the ContainerLogV2 schema

Control 07.20

Priority: Low

Consider spot node pools for non time-sensitive workloads

> **Applicability:** Applies when existing workloads use Spot or restartable, eviction-tolerant workloads justify Spot capacity; regular capacity is valid otherwise.

Use Spot capacity for restartable batch jobs and other eviction-tolerant workloads, with checkpointing, taints, and explicit scheduling. There is no capacity guarantee; keep system and availability-critical workloads on regular capacity and design fallback when Spot allocation fails.

DOCUMENTATION

- Add an Azure Spot node pool to an AKS cluster

Control 07.21

Priority: High

Configure alerts on the most critical metrics (see Container Insights for recommendations)

Configure actionable Prometheus, platform metric, and log alerts for workload availability, node readiness, pending pods, resource saturation, and failed upgrades. Route to owned action groups, test delivery and runbooks, and tune thresholds to SLOs and expected scaling behavior.

DOCUMENTATION

- Recommended alert rules for Kubernetes clusters

Control 07.22

Priority: Medium

Monitor SNAT capacity on the actual outbound path

SNAT exhaustion causes intermittent outbound connection failures. Monitor used and allocated SNAT ports and failed connections on the component providing outbound access: Standard Load Balancer, NAT Gateway, Azure Firewall, or another NVA. Using a firewall doesn't remove the need to monitor SNAT.

DOCUMENTATION

- Use a public standard load balancer in Azure Kubernetes Service

Control 07.23

Priority: High

Subscribe to resource health notifications for your AKS cluster

Configure Azure Resource Health and Service Health alerts for relevant resources and regions, with owned action groups. Correlate platform incidents with workload telemetry; platform health notifications don't replace application availability checks.

DOCUMENTATION

- Create Resource Health alerts

Control 07.24

Priority: Low

Consider subscribing to EventGrid Events for AKS automation

Subscribe to AKS Event Grid notifications for supported lifecycle events such as available upgrades. Make handlers idempotent and monitor delivery failures; an event is a trigger to evaluate readiness, not authorization to upgrade production immediately.

DOCUMENTATION

- AKS as an Event Grid source

Control 07.25

Priority: High

Protect the AKS-managed node resource group from direct changes

Manage nodes and related infrastructure through supported AKS APIs, not by editing their VM scale sets, network interfaces, or disks directly. Evaluate node resource group lockdown to enforce read-only access and avoid accidental changes; AKS Automatic preconfigures a fully managed node resource group.

DOCUMENTATION

- Node resource group lockdown

Control 07.26

Priority: High

Prepare a least-privilege, on-demand debugging runbook

Prepare approved diagnostic images, kubectl debug procedures, and AKS diagnostic collection before an incident. Prefer short-lived debugging sessions over permanently installed privileged toolboxes; restrict permissions, audit use, and redact secrets before sharing diagnostic bundles.

DOCUMENTATION

- Connect to AKS nodes for maintenance or troubleshooting

Control 07.27

Priority: Medium

Consider using Long-Term Support (LTS) for Kubernetes versions

> **Applicability:** Applies when extended Kubernetes validation cycles justify Premium LTS or the cluster is already enrolled; standard support remains valid when upgrades meet its lifecycle.

Evaluate Premium tier with the explicitly enabled AKSLongTermSupport plan when extended validation cycles justify the extra cost. AKS provides approximately one year of community support plus one year of LTS, totaling approximately 24 months from that Kubernetes version's GA, not 24 months from enrollment.

All supported Kubernetes versions are eligible; check the current release calendar and offered regional upgrade targets. LTS supports only the two latest patch versions, so retain patch upgrades and plan the next minor-version migration. OS images, add-ons, and extensions have separate lifecycles; LTS doesn't extend retired OS support. AKS Automatic's Standard tier defaults aren't equivalent to Premium LTS.

DOCUMENTATION

- Long-Term Support for AKS
- Supported Kubernetes versions in AKS

Control 07.28

Priority: High

Rehearse platform upgrades with maintenance and replacement strategies

Configure aksManagedAutoUpgradeSchedule for Kubernetes auto-upgrades and aksManagedNodeOSUpgradeSchedule for node OS updates. The default maintenance configuration controls weekly AKS component releases, not both upgrade channels. Allow at least four hours for automatic upgrades.

A maintenance schedule does not enable auto-upgrade and is not an absolute downtime boundary: urgent repairs and platform-initiated upgrades can occur outside it. Test readiness probes, replica placement, PDB drainability, surge capacity, subnet IPs, quota, drain timeout and soak time. For high-risk platform changes where in-place upgrade risk exceeds tolerance, evaluate a parallel-cluster blue/green replacement: validate state and data migration, external dependencies, DNS and traffic cutover, temporary double capacity/quota/cost, rollback criteria and a safe rollback window before decommissioning the old cluster. Kubernetes upgrades normally move between supported minor versions; parallel replacement is not a requirement for a presumed Kubernetes major upgrade. This is platform replacement, distinct from canary or blue/green application releases. Avoid promising zero disruption.

DOCUMENTATION

- Planned maintenance configuration types and limitations
- AKS upgrade mechanics and workload responsibilities
- Blue-green deployment of AKS clusters

Control 07.29

Priority: High

Inventory node OS lifecycles independently from Kubernetes support

Migrate Azure Linux 2.0 pools to a supported OS immediately: security support ended November 30, 2025 and node image removal began October 31, 2026, preventing scaling. Track each pool's actual OS image and OS SKU rather than assuming an LTS Kubernetes plan extends OS support.

Choose a supported Azure Linux, Ubuntu or other supported OS SKU from workload, driver and compliance needs; Azure Linux is not mandatory. Plan Ubuntu 20.04 migration before support and image removal on March 17, 2027, and Ubuntu 22.04 migration before support ends June 30, 2027 and image removal April 30, 2028. Azure Linux with OS Guard preview support ends December 10, 2026 with images removed March 10, 2027. Validate target OS, Kubernetes version, architecture, agents, drivers and security features, plus replacement-pool drains; generic versus versioned OS SKUs can have different upgrade behavior. For immutable nodes, evaluate Azure Container Linux against current compatibility requirements rather than assuming it supports every Azure Linux feature.

DOCUMENTATION

- AKS node images, OS support, and retirement dates
- Upgrade AKS node OS versions
- Use the Azure Linux host on AKS
- Node pool OS support and retirement notices

Control 07.30

Priority: High

Choose AKS Automatic or Standard separately from the support pricing tier

Evaluate AKS Automatic for supported workloads that benefit from managed node provisioning, upgrades, safeguards, and reduced platform operations. Choose AKS Standard when Windows, unsupported networking combinations, or platform customization require customer-managed configuration.

Automatic versus Standard describes the operating model, not the Free, Standard, or Premium cluster management pricing tier. Automatic preconfigures Standard tier, an API server uptime SLA, and a qualifying pod readiness SLA; neither guarantees application availability. Premium with the LTS plan is a separate choice for eligible Standard-mode clusters. Review fixed, default, and optional settings before selecting a mode, including the inability to stop NAP-enabled clusters.

DOCUMENTATION

- AKS Automatic feature comparison
- Free, Standard, and Premium pricing tiers

Control 07.31

Priority: Medium

Budget and tune observability collection without losing incident evidence

Set telemetry budgets and review log ingestion, retention, Prometheus cardinality, scrape frequency, and duplicate collection. Use data collection rules, ConfigMap filters, and supported metric profiles to retain the signals needed for security, SLOs, and investigations.

Choose log table plans and retention based on query, alerting, export, and compliance needs, not ingestion price alone. Validate dashboards and alerts after every collection change. Dropping audit records or verbose application logs can reduce cost but also remove evidence that can't be recreated later.

DOCUMENTATION

- Optimize monitoring costs for Kubernetes
- ContainerLogV2 configuration and table plan considerations

Control 07.32

Priority: High

Keep admission webhooks available and rehearse safe recovery

> **Applicability:** Applies when mutating or validating admission webhooks are installed, including policy, service mesh and extension webhooks.

Inventory admission webhook dependencies and operate highly available backends with tested API-server reachability, healthy endpoints and valid serving certificates/CA trust. Monitor admission latency and errors and verify that backend loss cannot unexpectedly block cluster recovery or silently bypass required policy.

Review timeoutSeconds and failurePolicy for each webhook: Fail protects enforcement but can block writes during an outage, while Ignore preserves availability at the cost of bypass risk. Scope namespace/object selectors and resource rules narrowly, especially for system components and the webhook's own recovery dependencies; document reviewed exclusions rather than broadly disabling controls. Test node drains, certificate rotation, network restrictions and Kubernetes upgrades, and rehearse a tightly authorized, audited emergency recovery procedure with restoration of enforcement.

DOCUMENTATION

- Validate AKS admission controllers and webhooks
- Kubernetes dynamic admission webhook configuration

Control 07.33

Priority: High

Validate resilience with controlled fault injection under load

> **Applicability:** Applies to workloads with availability or recovery targets that require evidence of behavior during component and dependency failures.

Run hypothesis-driven resilience tests under representative load, beginning in an isolated nonproduction environment. Measure availability, latency, recovery time and data integrity against workload SLOs while injecting bounded pod, node, network and dependency faults. Record failures and verify remediation.

Define an approved blast radius, steady-state metrics, abort thresholds, rollback/recovery steps, owners and observation coverage before each experiment. Include saturation, timeouts, retry amplification, scaling and dependency throttling where relevant. Production fault injection is optional and requires separate risk approval and proven safety controls; never require destructive production chaos or delete production to satisfy this check. Operational fault testing complements disaster-recovery rebuild and restore drills.

DOCUMENTATION

- Architecture strategies for reliability testing

ARCHITECTURE DOMAIN 08 / 11

Business continuity - disaster recovery

Test failure assumptions and prove that applications and data can meet recovery objectives.

15 review controls / 10 high / 3 medium / 2 low

Control 08.01

Priority: High

Define non-functional requirements such as SLAs, RTO (Recovery Time Objective) and RPO (Recovery Point Objective)

Define workload availability SLOs, maximum tolerable data loss (RPO), and recovery time (RTO) before production. Cover pod, node, zone, regional, deletion, and corruption scenarios along with external dependencies. Choose architecture, replication, backup frequency, retention, and recovery capacity from these targets and validate them with measured drills rather than service SLA percentages alone.

DOCUMENTATION

- Define the BCDR strategy

Control 08.02

Priority: High

Rehearse disaster recovery and clean-environment rebuilds regularly

Rebuild into an isolated recovery environment from infrastructure as code, restore representative backups, and validate application and data integrity. Do not delete production to test recovery. Rehearse node, zone, and regional failures appropriate to the workload, and measure achieved RPO and RTO.

Keep versioned runbooks with recovery order, owners, credentials and break-glass access, DNS and traffic changes, validation gates, rollback, and failback. Test without relying on the failed region's pipelines, registry endpoints, secrets, or monitoring, and record remediation actions after drills.

DOCUMENTATION

- Schedule and perform DR tests regularly

Control 08.03

Priority: High

Use Availability Zones if supported in your Azure region

> **Applicability:** Availability-sensitive workloads requiring resilience to an intra-region zone failure where the Azure region, node VM sizes, and dependencies support availability zones.

Where supported by region and VM SKU, distribute application node pools across availability zones and spread replicas across nodes and zones. Zones are separate datacenter failure domains within a region, not update domains in one datacenter. Align storage redundancy, load balancing, and recovery capacity; a zone-enabled cluster does not automatically make every workload or dependency zone-resilient.

DOCUMENTATION

- Create an AKS cluster across availability zones

Control 08.04

Priority: Medium

Plan for multiregion deployment

> **Applicability:** Only workloads whose availability, RPO, RTO, or regulatory requirements require recovery in another region; multiple live regions are not required for every workload.

Select active-active, warm standby, or cold recovery from workload RPO/RTO, consistency, cost, and data residency requirements. Validate AKS, VM, storage, network, and quota availability in each target region and provision failover capacity. Region pairing is not mandatory for all architectures, but specific replication and Azure Backup Cross Region Restore capabilities depend on supported paired regions.

DOCUMENTATION

- Plan for multiregion deployment

Control 08.05

Priority: High

Create a data replication, recovery, and failback plan

> **Applicability:** Stateful workloads and external data dependencies whose RPO and RTO require data recovery or failover across failure domains; regional replication is needed only when regional recovery targets justify it.

Inventory persistent volumes and external data services and define supported replication or backup/restore for each according to RPO and RTO. Validate consistency and replication lag against RPO, including write fencing to avoid split-brain during failover. Measure recovery and failback time in drills, including data reconciliation and application restart order. Replication can propagate deletion and corruption and is not a substitute for independently retained backups.

DOCUMENTATION

- AKS multi-region deployment models
- AKS backup and restore support matrix

Control 08.06

Priority: High

Use Standard or Premium tier for production control-plane uptime SLA

Use Standard tier for production cluster management, or Premium when long-term Kubernetes support is required. Standard and Premium include a Kubernetes API server uptime SLA of 99.95% with availability zones or 99.9% without, subject to current service terms. Free tier has no financially backed uptime SLA.

The control-plane SLA is not an end-to-end application availability guarantee. AKS Automatic is preconfigured with Standard tier and also has qualifying pod readiness SLA coverage under separate conditions; workload replicas, topology, storage, networking, and recovery still require design and testing.

DOCUMENTATION

- Azure Kubernetes Service (AKS) Uptime SLA
- Reliability in Azure Kubernetes Service

Control 08.07

Priority: High

Spread workload replicas across nodes and zones

> **Applicability:** Availability-sensitive workloads that need replicated serving capacity during node or zone failure; apply zone spreading where supported and required by the workload availability target.

Use topology spread constraints for kubernetes.io/hostname and topology.kubernetes.io/zone, or justified anti-affinity, to distribute replicas across failure domains. Match label selectors to the intended workload and ensure enough replicas and capacity remain after a domain failure.

Choose maxSkew and DoNotSchedule versus ScheduleAnyway deliberately. Strict rules can leave replacement pods pending when a zone or capacity is unavailable; test their interaction with volume topology, taints, autoscaling, and upgrades. Scheduling constraints do not automatically rebalance every already-running pod.

DOCUMENTATION

- Pod topology spread constraints

Control 08.08

Priority: Medium

If using a private registry (ACR), configure region replication to store images in multiple regions

> **Applicability:** Multi-region recovery scenarios where AKS depends on a private registry and required container images must remain accessible during a regional outage.

For multi-region AKS deployments, evaluate ACR Premium geo-replication or another tested image-distribution strategy so recovery nodes can pull required digests during a regional outage.

ACR geo-replication is asynchronous and is not an independent backup against deletion or compromise. Confirm required images and OCI artifacts are available before regional rollout, configure private endpoints and DNS for replicas where needed, and test cold-node pulls during failover.

DOCUMENTATION

- Enable geo-replication for container images

Control 08.09

Priority: Medium

Verify registry regional zone support and data-plane resilience

> **Applicability:** Deployments using Azure Container Registry where registry availability is part of the workload's zone-resilience requirements.

ACR data-plane zone redundancy is enabled by default for all tiers in regions supporting availability zones, including existing registries and supported geo-replicas. Verify region support rather than requiring the legacy zoneRedundancy property to be Enabled; that property no longer controls behavior. ACR Tasks do not currently support availability zones, and zone redundancy does not protect against a region-wide outage.

DOCUMENTATION

- Zone redundancy in Azure Container Registry

Control 08.10

Priority: Low

Separate the registry lifecycle from disposable compute resources

> **Applicability:** Shared or recovery-critical registries whose artifacts must outlive disposable AKS environments.

Place shared registries in a resource group whose lifecycle is independent of disposable AKS environments. Apply least-privilege management access and appropriate resource locks to reduce accidental registry deletion.

Resource groups are lifecycle and management scopes, not backup boundaries. Registry resource locks do not prevent all data-plane artifact deletion; protect repository permissions and retain recoverable copies of critical release artifacts.

DOCUMENTATION

- Registry best practices - Use dedicated resource group

Control 08.11

Priority: Low

Evaluate ACR soft delete preview and compatible artifact recovery controls

> **Applicability:** Azure Container Registry deployments evaluating artifact deletion recovery where preview support terms and soft-delete compatibility limitations are acceptable.

Where preview use is acceptable, evaluate ACR soft delete with a tested 1-90 day retention and restore process. It currently does not support geo-replicated registries or artifact cache and cannot coexist with the retention policy. Retained artifacts incur storage charges. Use another compatible recovery strategy when these limitations conflict with regional resilience requirements.

Restore tests must include multi-architecture manifests and OCI referrers: restoring a manifest list or subject does not recursively restore child manifests or signatures. Soft delete is not an independent backup against registry deletion or compromise.

DOCUMENTATION

- Enable soft delete policy

Control 08.12

Priority: High

Select and test a supported recovery solution for cluster resources and data

> **Applicability:** Workloads with Kubernetes resources or durable data that must be recovered after loss, deletion, or corruption. Select coverage per volume type and external data service; Azure Backup-specific requirements apply only when that implementation is selected.

Select Azure Backup for AKS or another supported recovery solution that meets measured RPO, RTO, retention, and consistency requirements. Inventory Kubernetes resources, PV types, excluded data, and managed external data services; configure supported protection for each and test restores for data integrity and application readiness. Protect backup resources with a lifecycle independent of disposable clusters and source data. Document dependencies on snapshots, metadata, source accounts, and regions rather than treating replication or successful backup jobs as proof of recoverability.

When using Azure Backup, Operational backups use snapshots and extension metadata storage in your subscription; eligible Azure Disk backups can also transfer to the Vault tier, but a Backup vault does not mean all backup data is vaulted. Check the live support matrix: Azure Files SMB requires CSI driver 1.32+, is Operational-tier only with at most 30-day retention, and has network, file-count, and restore-location restrictions. NFS Files, Blob CSI, Azure Container Storage, and in-tree PVs are not covered. Enable CSI snapshots, supported managed identities, a healthy Linux backup extension, Trusted Access, and required networking/RBAC; don't co-install Velero with that extension. Snapshot backups are crash-consistent, not automatically application-consistent or atomic across volumes; use supported application hooks or database-native protection when needed. Protect excluded resources and external services separately and validate that source deletion cannot remove the only recovery copy.

DOCUMENTATION

- Azure Backup for AKS
- Back up Azure Kubernetes Service using Azure Backup
- AKS backup support matrix and limitations

Control 08.13

Priority: High

Match backup tiers and cross-region restore to measured RPO and RTO

> **Applicability:** Only deployments selecting Azure Backup for AKS; assess Vault tier and Cross Region Restore only when their supported scenarios are needed to meet recovery requirements.

Validate backup cadence, completion, tiering, replication lag, and restore duration against recovery targets. Operational AKS backups have a minimum scheduled interval of four hours and restore within the source region. Vault-tier recovery points are daily for eligible Azure Disks; Cross Region Restore requires supported paired regions, a geo-redundant Backup vault, and Cross Region Restore enabled.

The current matrix describes up to 24-hour primary-region and up to 36-hour secondary-region vaulted RPO, not continuous point-in-time recovery. Vaulting is region-limited and currently supports at most 100 Azure Disk PVs per backup instance with disks no larger than 1 TB, public disk network access enabled, and no Disk Encryption Set or Microsoft Entra data-access authentication. Azure Files is not eligible. Do not weaken required security controls simply to enable vaulting; choose another supported recovery strategy when necessary. Test target-cluster capacity, compatible APIs, identities, networking, recovery order, and cleanup of staged restore resources.

DOCUMENTATION

- Vaulted backup and Cross Region Restore limitations

Control 08.14

Priority: High

Protect recovery points from deletion and monitor restore readiness

> **Applicability:** Only deployments using Azure Backup for AKS. For other recovery implementations, assess equivalent deletion protection, monitoring, and restore readiness under the recovery-solution control.

Separate backup administration from workload administration and evaluate supported vault immutability, enhanced soft delete, and multi-user authorization. Alert on backup failures, skipped volumes, stale recovery points, and vault-tier transfer failures. Validate restore permissions and recovery-point usability through drills.

Vault protections do not automatically protect Operational-tier snapshots or extension metadata in your subscription. Azure Files snapshots depend on the source share and account lifecycle. Confirm supported resource-lock behavior, deletion permissions, and retention for each storage type rather than treating successful configuration as ransomware protection.

DOCUMENTATION

- AKS backup supported protection features and limitations

Control 08.15

Priority: High

Configure disruption budgets and test maintenance capacity

> **Applicability:** Replicated availability-sensitive workloads managed through the Eviction API during drains, upgrades, or consolidation; not a blanket requirement for restartable batch jobs or workloads that deliberately accept interruption.

Create separate policy/v1 PodDisruptionBudget objects with selectors matching replicated services and tested minAvailable or maxUnavailable values. Verify healthy replicas, allowed disruptions, and spare scheduling capacity so drains and upgrades can progress within the workload availability target. Test eviction behavior with unhealthy pods and measure drain completion time; configure readiness, rollout strategy, replicas, and topology separately.

PDBs constrain voluntary Eviction API operations such as normal node drains, not hardware or zone failures, direct pod deletion, or Deployment and StatefulSet rolling updates. Involuntary disruptions can reduce the remaining disruption allowance but aren't prevented by the budget. A zero-disruption budget can block maintenance; permit at least one eviction where the workload allows it and document coordination when it doesn't. Consider unhealthyPodEvictionPolicy: AlwaysAllow when an unhealthy pod would otherwise block draining, and test that policy against application recovery behavior. A PDB isn't an uptime guarantee.

RELATED CONTROLS

- Implement a proper Readiness probe
- Implement graceful termination and use preStop hooks when needed
- Spread workload replicas across nodes and zones

DOCUMENTATION

- Limit disruption impact with pod disruption budgets
- Specifying a disruption budget for your application

ARCHITECTURE DOMAIN 09 / 11

Windows

Review the support, security and operational requirements specific to Windows workloads.

14 review controls / 6 high / 7 medium / 1 low

Control 09.01

Priority: High

Map the base image to node OS

> **Applicability:** Only deployments running or planning Windows container workloads on Windows node pools.

Use supported Windows Server 2022 or 2025 node images and validate the official host/container compatibility matrix. AKS supports process isolation, not Hyper-V isolation; don't assume all container image versions can run on every Windows host.

Windows Server 2019 support ended March 1, 2026 and it isn't supported on Kubernetes 1.33 or later. Select compatible node OS, Kubernetes, and container base image versions together, and use OS/build-aware scheduling when multiple Windows versions coexist.

DOCUMENTATION

- Windows container version compatibility
- Limitations of Windows containers

Control 09.02

Priority: Medium

Test Windows application shutdown and interrupted-work recovery

> **Applicability:** Only Windows container workloads whose shutdown and interrupted-work recovery must be validated during drains or termination.

Don't assume Linux signal-handling behavior applies to Windows. Test how the Windows runtime and application respond to termination, lifecycle hooks, and terminationGracePeriodSeconds during a real node drain. Choose sufficient shutdown time, stop accepting new requests, and make interrupted work recoverable.

DOCUMENTATION

- Understand pod lifecycle
- Limitations of Windows containers

Control 09.03

Priority: Medium

Don't use privileged containers

> **Applicability:** Only deployments running ordinary Windows application containers; assess trusted host-level administration separately under the HostProcess control.

Ordinary Windows containers don't support Linux privileged-container mode. Use ContainerUser where the application permits and restrict host access. Windows HostProcess containers are a separate, highly privileged mechanism for trusted node administration, not a way to run ordinary application workloads.

DOCUMENTATION

- Limitations of Windows containers

Control 09.04

Priority: Medium

Watch for memory usage

> **Applicability:** Only deployments running Windows nodes and container workloads.

Windows memory accounting, paging, and node-pressure behavior differ from Linux; don't rely on Linux OOM and eviction assumptions. Set measured memory requests and limits, monitor committed memory and paging as well as working set, and retain headroom for the OS and system services.

DOCUMENTATION

- Limitations of Windows containers

Control 09.05

Priority: High

Choose a Windows-compatible Azure CNI network model

> **Applicability:** Only clusters with Windows node pools or a planned migration to Windows workloads.

Windows node pools require Azure CNI; kubenet and bring-your-own CNI aren't supported. Evaluate Azure CNI Overlay for address-space efficiency or a supported VNet-routable mode when direct pod addressing is required.

Check the Windows OS, Kubernetes version, and network policy engine compatibility together. Azure CNI powered by Cilium doesn't support Windows; a Linux-only networking recommendation must not be applied blindly to mixed-OS clusters.

DOCUMENTATION

- What network plug-ins are supported?

Control 09.06

Priority: High

Patch Windows hosts and rebuild container base images independently

> **Applicability:** Only deployments maintaining Windows node images or Windows container base images.

AKS publishes Windows node images monthly; Windows Update doesn't automatically patch these hosts. Apply node image upgrades through NodeImage automation or a monitored manual process, and separately rebuild and redeploy application images from patched Windows base images.

Node image upgrades don't update your application images. Test host/base-image compatibility and rolling drains before production, and don't assume matching monthly revision numbers are universally required across supported Windows versions. SecurityPatch isn't supported for Windows; Unmanaged behaves like None.

DOCUMENTATION

- Windows container version compatibility
- Windows node monthly updates

Control 09.07

Priority: High

Secure the traffic of your containers

> **Applicability:** Only deployments running Windows workloads that require pod network isolation; affected Windows Network Policy Manager deployments must migrate to a supported implementation.

Apply least-privilege pod network policies with an engine supported for the exact Windows OS and network model, and verify enforcement with connectivity tests. Use application authentication and TLS as additional controls, not substitutes for network isolation.

Azure Network Policy Manager on Windows became unsupported on September 30, 2026. Cilium isn't supported on Windows; use the current AKS policy support matrix to evaluate Calico and validate any OS-version gaps before migration.

DOCUMENTATION

- AKS network policy engine support and limitations

Control 09.08

Priority: Medium

Use gMSA when Windows workloads require Active Directory authentication

> **Applicability:** Only Windows workloads requiring Active Directory domain authentication through a supported gMSA integration; workloads without this dependency do not need gMSA.

Use group Managed Service Accounts for applications that require Active Directory domain authentication. Configure the supported AKS integration, credential specifications, Key Vault access, and domain-controller connectivity with least privilege; don't enable gMSA for workloads that don't need it.

DOCUMENTATION

- Enable Group Managed Service Accounts (GMSA) for your Windows Server nodes

Control 09.09

Priority: Medium

If required for AKS Windows workloads, HostProcess containers can be used

> **Applicability:** Only Windows deployments with a justified host-level administration or node-agent requirement that needs HostProcess access; not ordinary application containers.

Use Windows HostProcess containers only for trusted host-level administration such as node agents. They run in the host context with access beyond ordinary container isolation; constrain who can deploy them, use approved images, and audit their permissions and activity.

DOCUMENTATION

- Use Windows HostProcess containers

Control 09.10

Priority: Low

Taint Windows nodes

> **Applicability:** Only clusters containing Windows pools, including mixed-OS deployments where workloads must be scheduled onto compatible hosts.

Set kubernetes.io/os: windows selectors for Windows workloads and Linux selectors for Linux workloads; use Windows build labels or RuntimeClasses when container compatibility requires a specific host build. Taint Windows pools and grant tolerations only to intended workloads; a toleration alone doesn't select a Windows node.

DOCUMENTATION

- Adapt apps for mixed-OS Kubernetes clusters using node selectors or taints and tolerations

Control 09.11

Priority: Medium

Validate Calico policy support before migrating Windows network enforcement

> **Applicability:** Only Windows deployments using or evaluating AKS-managed Calico, including migrations from retired Windows Network Policy Manager; not a requirement to adopt Calico on other clusters.

AKS-managed Calico supports Kubernetes networking.k8s.io policies on documented Windows configurations. Validate the current OS/network compatibility matrix and test ingress, egress, DNS, and service traffic before replacing an existing policy engine.

The current network policy matrix lists Windows Server 2019 and 2022 for Calico; that doesn't restore support for retired Windows Server 2019 or establish support for Windows Server 2025. Verify the latter before migrating. AKS-managed Calico doesn't support projectcalico.org policies or running alongside Calico Cloud or Enterprise.

DOCUMENTATION

- Secure traffic between pods using network policies

Control 09.12

Priority: Medium

Verify Accelerated Networking on supported Windows VM sizes

> **Applicability:** Only Windows node pools using VM sizes, images, and drivers that support Accelerated Networking.

AKS enables Accelerated Networking on supported VM sizes. Verify VM, image, and driver support and benchmark network throughput and latency; don't modify AKS-managed NICs directly to force enablement on unsupported configurations.

DOCUMENTATION

- What is Accelerated Networking?

Control 09.13

Priority: High

Migrate retired Windows OS channels and plan supported LTSC upgrades

> **Applicability:** Only Windows deployments using retired OS channels or planning an LTSC migration before the documented support and image-removal deadlines.

Migrate Windows Server 2019, unsupported since March 1, 2026, before image removal on April 1, 2027. Windows Annual Channel preview support ended May 15, 2026, with image removal on May 15, 2027. Retired images eventually prevent scaling and remediation, not just security updates.

Evaluate Windows Server 2025 LTSC on Kubernetes 1.32 or later and test host/container compatibility, agents, gMSA, storage, and network policy first. Windows Server 2025 pools require FIPS-enabled images. Windows Server 2022 support ends June 30, 2028 and isn't available for Kubernetes 1.37 or later; images are removed June 30, 2029. A new-pool migration permits staged validation. The in-place 2022-to-2025 migration uses aks-preview and requires Kubernetes 1.32 to 1.36; assess its preview support terms before use.

DOCUMENTATION

- Windows container considerations and retirement dates
- Upgrade Windows OS versions

Control 09.14

Priority: High

Include Windows-specific capacity and observability in the operating model

> **Applicability:** Only clusters operating Windows user pools and their supporting Linux system pool, capacity, and monitoring integrations.

Maintain a Linux system pool alongside Windows user pools and include Windows startup, image-pull, patching, and drain times in capacity and maintenance tests. Windows pools aren't supported by NAP or AKS Automatic, so select a supported AKS Standard configuration.

For managed Prometheus, enable Windows recording rules and confirm Windows exporter metrics and Windows dashboards are populated. Monitor memory commitment, paging, storage, network errors, and application availability rather than relying only on Linux dashboards.

DOCUMENTATION

- Windows monitoring best practices
- Node auto-provisioning limitations

ARCHITECTURE DOMAIN 10 / 11

Application deployment

Translate platform capabilities into safe releases and resilient application behavior.

22 review controls / 7 high / 9 medium / 6 low

Control 10.01

Priority: High

Implement a proper Liveness probe

Use a liveness probe when restarting the container can recover an unhealthy application. Check local process health rather than shared downstream dependencies to avoid restart storms during an outage. Tune timeouts and failure thresholds under realistic load; failed liveness probes restart the container according to its restart policy, not the entire pod.

DOCUMENTATION

- Configure Liveness, Readiness and Startup Probes

Control 10.02

Priority: Medium

Implement a proper Startup probe

Protect slow-starting containers with a startup probe. Liveness and readiness probes do not run until it succeeds. Set failureThreshold and periodSeconds to cover measured worst-case startup time, and test cold starts; exhausting the startup failure threshold triggers container restart according to its restart policy.

DOCUMENTATION

- Configure Liveness, Readiness and Startup Probes

Control 10.03

Priority: High

Implement a proper Readiness probe

> **Applicability:** Applies to availability-sensitive services receiving traffic through Kubernetes Services or other readiness-aware routing.

Use readiness probes to indicate whether the application can serve traffic. A failed readiness probe normally removes the pod from ready Service endpoints without restarting its containers. Test dependency failures and recovery without making every replica unready for an optional dependency; account for endpoint propagation, existing connections, and Services that publish not-ready addresses.

DOCUMENTATION

- Configure Liveness, Readiness and Startup Probes

Control 10.04

Priority: High

Implement graceful termination and use preStop hooks when needed

> **Applicability:** Applies to availability-sensitive services or workers that must drain requests or checkpoint work during planned termination.

Handle termination signals, stop accepting new work, and drain in-flight requests or checkpoint work within terminationGracePeriodSeconds. Add a preStop hook only when needed and include its execution time in the grace period. Hooks and graceful shutdown are not guaranteed during node loss, force deletion, or abrupt process failure; test rolling updates and node drains.

DOCUMENTATION

- Container Lifecycle Hooks

Control 10.05

Priority: High

Run more than one replica for your Deployment

> **Applicability:** Applies to availability-sensitive services whose design permits concurrent replicas; singleton workloads require a separately tested recovery design.

Run multiple ready replicas for services that require availability during pod failure or maintenance. Size the replica count and autoscaler minimum for the availability target and remaining capacity after a failure. Spread replicas across nodes and, where available, zones; extra replicas on one node are not sufficient.

DOCUMENTATION

- Pod replicas

Control 10.06

Priority: Medium

Apply consistent Kubernetes labels and Azure resource tags

Apply recommended Kubernetes application labels and ownership/environment metadata for selectors, policy, operations, and cost allocation. Azure resource tags and Kubernetes labels are separate mechanisms and are not automatically synchronized. Keep secrets and personal data out of both.

Define technical, business and security metadata, including owner, environment, cost center and lifecycle. Apply Azure tags through supported AKS cluster and node pool APIs and review propagation and replacement behavior before updates; do not directly retag AKS-managed infrastructure to bypass those APIs. Define deliberate mappings to Kubernetes labels and validate selectors before changing labels used by controllers or policies.

DOCUMENTATION

- Recommended Labels
- Use Azure tags in Azure Kubernetes Service
- Azure tags and Kubernetes labels in AKS

Control 10.07

Priority: Low

Implement autoscaling of your applications (Horizontal pod autoscaler, KEDA)

Use HPA for resource, custom, or external metrics, or the managed KEDA add-on for event-driven scaling. Configure resource requests for utilization-based targets, minimum/maximum replicas, stabilization, and metric failure behavior. Load-test startup latency and downstream capacity; coordinate pod scaling with node capacity and quotas. Do not configure independent HPA and KEDA controllers for the same workload.

DOCUMENTATION

- Horizontal Pod Autoscaler

TOOLS

- Kubernetes Event-driven Autoscaling (KEDA)
- KEDA as AKS addon

Control 10.08

Priority: Medium

Use namespaces with explicit access and isolation controls

Use dedicated namespaces rather than the default namespace for managed workloads. Apply scoped RBAC, network policies, resource quotas, and admission controls to enforce boundaries. Namespaces alone do not provide network isolation or a strong security boundary against hostile tenants; use separate clusters when the threat model requires them.

Name namespaces for application and environment ownership, reserve system namespaces for platform components, and verify that access, policy enforcement and quota behavior match the intended boundary. This control implements namespace protections after the separate tenant-isolation architecture decision.

DOCUMENTATION

- Namespaces

Control 10.09

Priority: Medium

Specify the security context of your pod/container

For Linux workloads, run as non-root, disallow privilege escalation, drop all capabilities and add back only justified exceptions, use RuntimeDefault seccomp, and make the root filesystem read-only where compatible. Avoid privileged containers, host namespaces, and hostPath mounts. Apply equivalent OS-specific controls for Windows rather than assuming Linux settings apply.

Use dedicated least-privilege service accounts and set automountServiceAccountToken: false when Kubernetes API credentials are unnecessary. This does not replace the explicit projected token used by Microsoft Entra Workload ID. Provide bounded writable volumes where the application requires them and document reviewed exceptions.

DOCUMENTATION

- Configure a Security Context for a Pod or Container

Control 10.10

Priority: Medium

Ensure your manifests respect good practices

Render Helm or Kustomize manifests in CI, validate their schemas and supported API versions against the target AKS version, and check security, requests, probes, and deployment policies before promotion. Use server-side dry runs where safe and enforce critical requirements with admission policy rather than relying only on developer linting.

Test rendered output against a representative target cluster to catch admission, permissions and scheduling errors. A formatter improves readability but is not evidence that APIs are supported or that the deployment is safe.

DOCUMENTATION

- Kubernetes configuration best practices
- Azure Policy for Kubernetes
- Manage Kubernetes objects declaratively

TOOLS

- kube-score
- Checkov
- kubelinter

Control 10.11

Priority: High

Conduct Dockerfile scanning to ensure Docker Image Security Best Practices

Define and lint a container build baseline: trusted maintained base images, non-root execution, multi-stage builds, minimal runtime contents, and no embedded credentials. Use build-time secret mounts instead of copying secrets into layers; deleting them in a later layer does not remove them from image history.

DOCUMENTATION

- AKS container image management best practices
- Kubernetes application security checklist

TOOLS

- Dockle
- Hadolint

Control 10.12

Priority: High

Scan built container images in CI before promotion

Scan the final image digest, including OS and application dependencies, before promotion and retain the result with the release. Integrate scanning into CI; registry or runtime scanning alone is not a synchronous build gate and cannot establish that an image is vulnerability-free.

DOCUMENTATION

- Microsoft Defender for Containers
- Container security in the software supply chain

Control 10.13

Priority: Medium

Enforce risk-based vulnerability gates for image releases

Gate image promotion using documented vulnerability severity, exploitability, exposure, and remediation deadlines. Record time-limited exceptions with owners and compensating controls.

Do not automatically allow a critical vulnerability merely because no vendor fix exists. Require explicit risk acceptance, consider replacing or removing the vulnerable component, and rescan retained and deployed image digests as new advisories appear.

DOCUMENTATION

- Container vulnerability management

Control 10.14

Priority: High

Compliance enforcement of Docker Image Builds

Evaluate image configuration and rendered deployment manifests against a documented security baseline before release, and enforce critical runtime settings through admission policy.

Check non-root execution, privilege escalation, capabilities, host access, and approved image sources. An exposed port alone does not establish a compliance violation, and Azure Policy admission checks do not replace Dockerfile or build-time scanning.

DOCUMENTATION

- Azure Built-In Policy
- Pod Security Standards

Control 10.15

Priority: Low

Assess application readiness before containerization

Assess runtime and operating-system support, external dependencies, state, licensing, health endpoints, and shutdown behavior before moving an application into containers. Select currently supported migration or build tooling for the application stack; automated repackaging does not establish production readiness or remove the need for testing.

DOCUMENTATION

- AKS architecture best practices

Control 10.16

Priority: Low

Apply the right deployment type to your application

Choose Deployment for replaceable services, StatefulSet for workloads requiring stable identity and storage, and Job or CronJob for finite or scheduled work. Configure rolling update surge/unavailable settings, readiness, rollout deadlines, and rollback procedures to match the workload and available capacity.

DOCUMENTATION

- Kubernetes workload management

Control 10.17

Priority: Low

Don't use naked pods

Manage production pods with a Deployment, StatefulSet, DaemonSet, Job, or other appropriate controller. Standalone pods are not recreated on another node after node loss.

Choose the controller according to lifecycle, identity, and storage requirements; a controller recreating a pod does not by itself recover data or guarantee service availability.

DOCUMENTATION

- Kubernetes workload controllers

Control 10.18

Priority: Low

Control the usage of imagePullPolicy

Set imagePullPolicy deliberately and deploy immutable digests rather than mutable latest tags. Always resolves the image reference on each start but can reuse cached layers; IfNotPresent can run a locally cached image. Test registry outages and new-node image pulls; digest pinning improves reproducibility but does not remove registry availability requirements.

DOCUMENTATION

- Updating images

Control 10.19

Priority: Medium

Use version-controlled infrastructure as code for Azure resources

Provision AKS and its dependencies with reviewed, version-controlled Bicep, ARM, or Terraform. Secure state and deployment identities, pin supported providers and modules, detect drift, and test repeatable environment recreation. Keep credentials out of templates and repositories.

DOCUMENTATION

- Create a Kubernetes cluster with Azure Kubernetes Service using Terraform

Control 10.20

Priority: Medium

Use canary or blue/green deployments

> **Applicability:** Applies when application-release risk and availability targets require progressive traffic shifting or parallel application versions.

Use canary or blue/green application releases where the risk justifies additional capacity and routing complexity. Define health and business-metric gates, progressive traffic shifts, rollback, and backward-compatible database changes. These strategies reduce release risk but do not guarantee availability during cluster upgrades.

DOCUMENTATION

- Safe deployment practices

Control 10.21

Priority: Low

Evaluate Dapr when its building blocks fit the application

> **Applicability:** Applies when Dapr is installed or an assessed application requirement justifies its building blocks; Dapr adoption is not a readiness requirement.

Consider Dapr for service invocation, pub/sub, state, or other supported building blocks when it reduces application complexity. Evaluate sidecar resources, component authentication, mTLS, supported versions, operations, and failure modes; adopting Dapr is optional and does not ensure uptime during cluster upgrades.

DOCUMENTATION

- Dapr project page

Control 10.22

Priority: Medium

Design retries, timeouts, and idempotency for dependency failures

Set bounded request timeouts, retry budgets with exponential backoff and jitter, and circuit breakers where appropriate. Make retried operations idempotent and support graceful degradation. Test dependency throttling and outages so retries and autoscaling do not amplify failure.

DOCUMENTATION

- Retry pattern

ARCHITECTURE DOMAIN 11 / 11

Image management

Protect the image supply chain from build and registry through admission and runtime.

9 review controls / 5 high / 3 medium / 1 low

Control 11.01

Priority: High

Scan the container image against vulnerabilities

Scan OS packages and application dependencies in built, stored, and deployed image digests, and rescan as vulnerability intelligence changes. Assign remediation owners and deadlines and rebuild affected images. Scanning identifies known issues; a clean result is not proof that an image is secure.

DOCUMENTATION

- Microsoft Defender for Containers
- Container image management best practices

TOOLS

- Microsoft Defender for Cloud
- Anchore
- Clair

Control 11.02

Priority: Medium

Allow deploying containers only from known registries

Enforce approved registry and repository paths with admission policy, including init and ephemeral containers. Restrict who can push or import into those repositories and combine source allowlists with digest and signature verification. Egress filtering is defense in depth, not a substitute for admission enforcement because cached images can run without a pull.

DOCUMENTATION

- Use the Azure Policy : Ensure only allowed container images in AKS
- Enforce supported admission policies with Azure Policy for AKS
- Using egress lockdown and authorizing only the URL of your registry

Control 11.03

Priority: High

Block promotion and redeployment of images with newly discovered issues

Continuously reassess image digests after publication and block promotion or new deployment when they violate policy. Use a supported admission and registry-access workflow rather than assuming a registry scan automatically quarantines images.

Track affected running workloads and cached images, rebuild and redeploy fixed digests, and document emergency exceptions. Blocking new image pulls does not stop existing containers or prevent cached images from being reused; coordinate containment with incident response.

DOCUMENTATION

- Container image management

Control 11.04

Priority: High

Use least-privilege identity-based access to container registries

Use separate identities for image publishing and pulling; disable the ACR admin account unless explicitly required. Grant the AKS kubelet managed identity pull-only permissions, not the application workload identity. Use AcrPull for registry RBAC mode; ABAC-enabled registries require Container Registry Repository Reader and do not support az aks --attach-acr. Scope repository permissions where supported.

DOCUMENTATION

- Azure Container Registry roles and permissions
- Azure Container Registry roles and permissions

Control 11.05

Priority: High

Enforce the required container registry network access boundary

> **Applicability:** Applies when the registry threat model or network policy requires private connectivity or restricted public endpoints; private endpoint adoption alone is not a readiness requirement.

Enforce the registry access boundary required by the threat model. For private access, use ACR Premium Private Link, configure private DNS and connectivity to registry and regional data endpoints, and disable public network access after validation. If private connectivity is not required, select an appropriate SKU with explicitly authorized public access rather than requiring Premium solely to pass this check.

Test image pulls from every node pool and DR region, plus CI builds and imports. Verify permitted and denied paths and that no public bypass remains where public access is prohibited. A private endpoint alone neither disables public access nor replaces identity-based authorization.

DOCUMENTATION

- Azure Container Registry Private Link

Control 11.06

Priority: Medium

Use minimal, maintained runtime images

Use a supported minimal or distroless runtime image appropriate for the application, architecture, and OS. Remove build tools and unused packages, run as non-root, and rebuild regularly from maintained bases.

Distroless images reduce attack surface but do not prevent exploitation or container escape. Validate certificates, runtime dependencies, patch availability, and a controlled debugging workflow rather than adding a shell to the production image.

DOCUMENTATION

- Google distroless images

Control 11.07

Priority: Low

Refresh container when base image is updated

Track base-image and dependency updates, including digest-pinned bases, and automate rebuilds followed by tests, scanning, signing, and staged redeployment. Updating a base image or rebuilding an application does not patch already-running containers; promote and deploy the new digest and retain a tested rollback image.

DOCUMENTATION

- Automatically build new images on base image update
- ACR Tasks base-image update triggers

Control 11.08

Priority: High

Sign OCI artifacts and verify trusted publishers before deployment

Sign released image digests and other OCI artifacts using supported tooling such as Notation. Verify integrity and trusted publisher identity in build and deployment workflows, and enforce verification at admission where supported. Define trust policies, certificate rotation, revocation response, and narrowly scoped exceptions; merely storing a signature in ACR does not enforce trust.

DOCUMENTATION

- Overview of signing and verifying OCI artifacts

Control 11.09

Priority: Medium

Retain SBOMs and build provenance linked to immutable image digests

Generate software bills of materials and build provenance for releases, associate them with the exact OCI image digest, and retain them with signatures and scan evidence. Record source revision, builder identity, and dependencies for incident investigation. Protect build identities and verify evidence before promotion; an SBOM alone does not establish image integrity or absence of vulnerabilities.

DOCUMENTATION

- OCI artifact signing and verification
- Manage OCI artifacts in ACR

APPENDIX A / SUPPORTING RESOURCES

Reference guidance (not scored)

Useful conventions and tools to support the review. These references are not readiness requirements and never affect the score.

Reference A.01 Reference guidance (not scored)

Customize the name of the MC_ resource group

Reference guidance: if naming policy requires it, choose a custom node resource group name when creating the cluster. The name cannot be changed afterward; AKS manages this group's infrastructure regardless of its name. The supported MC_ default is not a readiness deficiency.

DOCUMENTATION

- Custom resource group name

Reference A.02 Reference guidance (not scored)

Reference: maintained Kubernetes operations tools

Reference catalog, not a readiness control: select maintained Kubernetes tools that fit the team's workflows. Review release activity, supply-chain provenance, kubeconfig access and compatibility before adoption; desktop tools and plugins can inherit powerful cluster permissions. No particular tool is required to complete the checklist.

TOOLS

- kubectl
- Helm
- kubectl aliases
- kubectx
- k9s